☕ Spring Framework: Real-World Case Studies - Complete Guide

JAVA 21 SPRING 7.0.3 MAVEN CASE STUDIES

Master Guide: Spring Framework Real-World Applications

Crafted with ❤ for Practical Spring Mastery

GITHUB AVINASH-706

CONTACT ME VIA GMAIL

> **Learning Note:** This comprehensive guide demonstrates three real-world Spring applications showcasing advanced Dependency Injection patterns, @Primary/@Qualifier resolution, @Lazy initialization, Bean Scopes, and Lifecycle management. Master production-ready Spring patterns through practical examples.

> **Prerequisites:** - Understanding of Spring IoC Container - Knowledge of Dependency Injection fundamentals - Familiarity with @Component, @Autowired, @Configuration - Bean Lifecycle and Scopes concepts

---

📚 Table of Contents

---

1. OVERVIEW

> 📝 **Author:** Avinash Dhanuka | GitHub

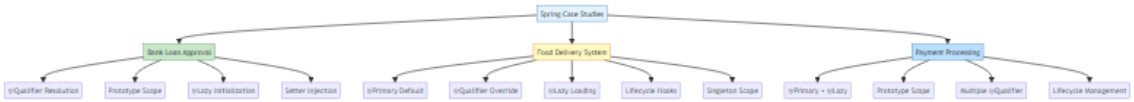
📌 What Are These Case Studies?

These three production-ready Spring applications demonstrate how advanced Spring concepts work together in real-world scenarios. Each case study solves a specific business problem while showcasing different Spring features.

🎯 Learning Objectives

- ✅ Master @Primary and @Qualifier for dependency resolution
- ✅ Understand @Lazy initialization for performance optimization
- ✅ Apply Singleton and Prototype scopes correctly
- ✅ Implement Bean Lifecycle with @PostConstruct and @PreDestroy
- ✅ Use Constructor and Setter injection appropriately
- ✅ Resolve ambiguity when multiple beans exist
- ✅ Build production-ready Spring applications

🏛️ Case Studies Architecture



🔍 Why These Case Studies Matter

Real-World Relevance: - Bank Loan Approval → Financial services, validation systems - Food Delivery → E-commerce, notification systems, order processing - Payment Processing → Payment gateways, transaction management

Spring Concepts Demonstrated: - Dependency resolution with multiple implementations - Performance optimization with lazy loading - Resource management with scopes - Lifecycle management for cleanup

---

2. CASE STUDIES SUMMARY

🎁 Projects Overview Table

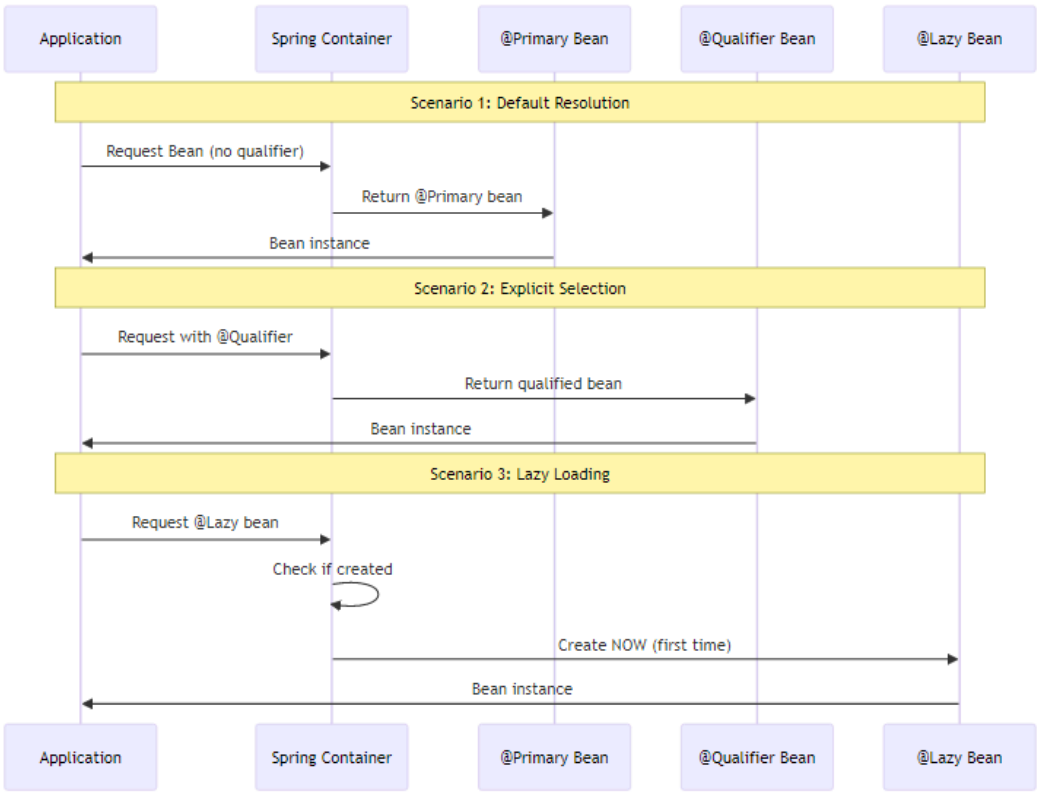
Case Study	Domain	Key Concepts	Complexity	Lines of Code
BankLoanApproval	Finance	@Qualifier, Prototype, @Lazy, Setter DI	⭐ ⭐ ⭐	~150
FoodDelivery	E-Commerce	@Primary, @Qualifier, @Lazy, Lifecycle	⭐ ⭐ ⭐ ⭐	~200
PaymentProcessing	FinTech	@Primary+@Lazy, Prototype, Multiple @Qualifier	⭐ ⭐ ⭐ ⭐	~180

🎯 Concepts Comparison Matrix

Concept	BankLoanApproval	FoodDelivery	PaymentProcessing
@Primary	✅ CreditScoreValidator	✅ EmailNotification	✅ CreditCardPayment
@Qualifier	✅ incomeValidator	✅ smsNotification	✅ upiPayment + creditCardPayment
@Lazy	✅ AuditService	✅ SmsNotification	✅ CreditCardPayment
Singleton	✅ LoanService	✅ DeliveryService	✅ TransactionLogger
Prototype	✅ IncomeValidator	❌	✅ UpiPayment
@PostConstruct	✅ AuditService	✅ DeliverySer-	✅ TransactionLog-

Concept	BankLoanApproval	FoodDelivery	PaymentProcessing
		vice	ger
@PreDestroy	✅ AuditService	✅ DeliverySer-vice	✅ TransactionLog-ger
Constructor DI	✅ LoanService	✅ OrderService	✅ PaymentProces-sor
Setter DI	✅ LoanService	✅ RestaurantSer-vice	❌

📊 Dependency Resolution Flow



3. CASE STUDY 1: BANK LOAN APPROVAL

📌 Business Problem

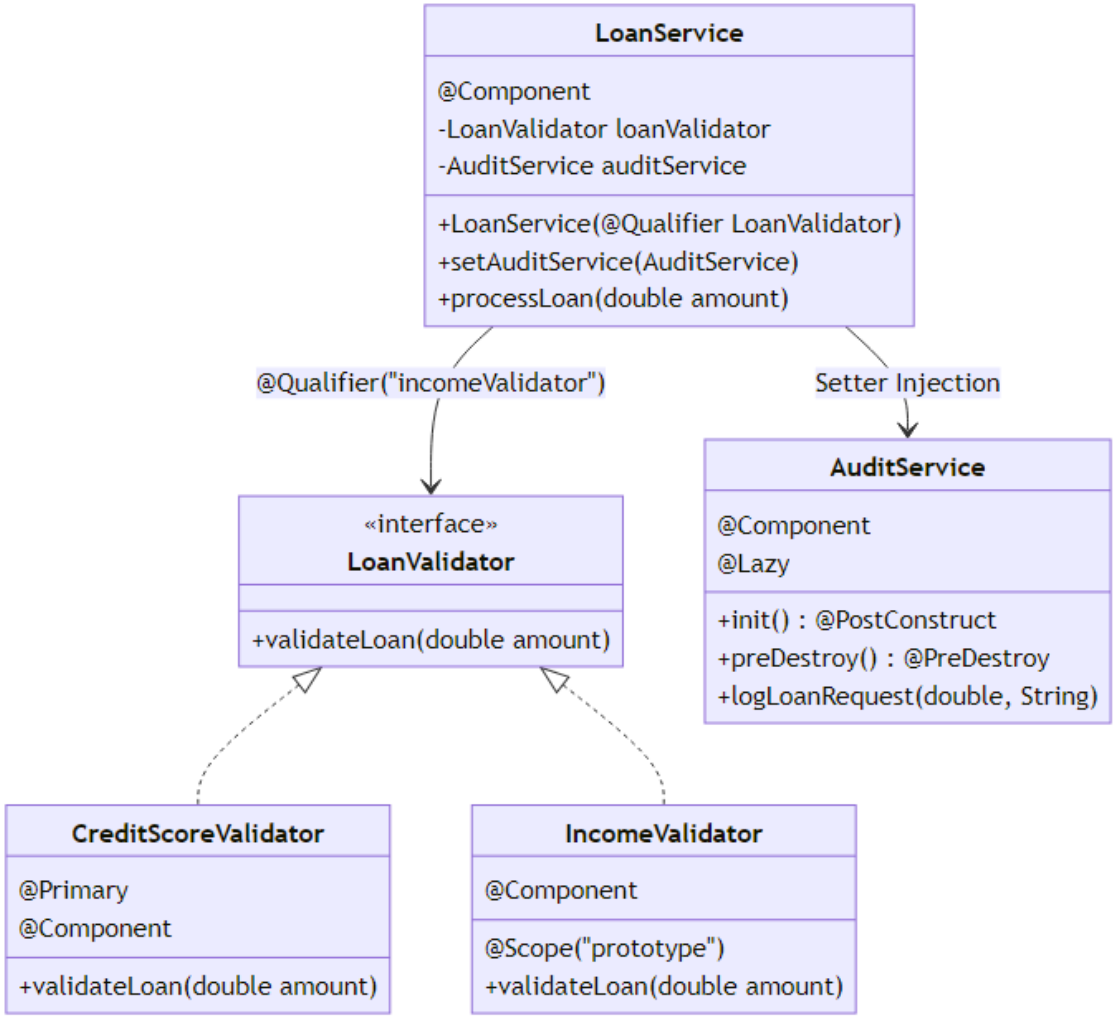
A bank needs a loan approval system that can validate loan applications using different validation strategies: - **Income-based validation** (check applicant's income) - **Credit score validation** (check credit history)

The system must be flexible to switch validators and audit all loan requests.

🎯 Spring Concepts Demonstrated

1. **@Qualifier** - Explicitly select IncomeValidator over CreditScoreValidator
2. **@Primary** - CreditScoreValidator as default validator
3. **Prototype Scope** - New IncomeValidator instance per request
4. **@Lazy** - AuditService loaded only when needed
5. **Setter Injection** - AuditService injected via setter
6. **Constructor Injection** - LoanValidator injected via constructor
7. **@PostConstruct/@PreDestroy** - AuditService lifecycle management

🏛 Architecture Diagram



🔍 Component Breakdown

1. LoanValidator Interface

```java
public interface LoanValidator {
    void validateLoan(double amount);
}
```

Purpose: Contract for different validation strategies

2. CreditScoreValidator (@Primary)

```
@Component
@Primary
public class CreditScoreValidator implements LoanValidator {
    @Override
    public void validateLoan(double amount) {
        System.out.println("CreditScoreValidator: Checking credit score for
         loan of $" + amount);
    }
}
```

Key Points: - Marked with @Primary → Default validator - Singleton scope (default) - Used when no @Qualifier specified

3. IncomeValidator (Prototype)

```
@Component
@Scope("prototype")
public class IncomeValidator implements LoanValidator {
    @Override
    public void validateLoan(double amount) {
        System.out.println("IncomeValidator: Checking income for loan of $"
         + amount);
    }
}
```

Key Points: - Prototype scope → New instance per request - Must use @Qualifier to inject - Useful for stateful validators

4. AuditService (@Lazy)

```
@Component
@Lazy
public class AuditService {
    @PostConstruct
    public void init() {
        System.out.println("AuditService initialized");
    }

    @PreDestroy
    public void preDestroy() {
        System.out.println("AuditService destroyed");
    }
```

```
    public void logLoanRequest(double amount, String validator) {
        System.out.println("AUDIT: Loan request for $" + amount + " using "
         + validator);
    }
}
```

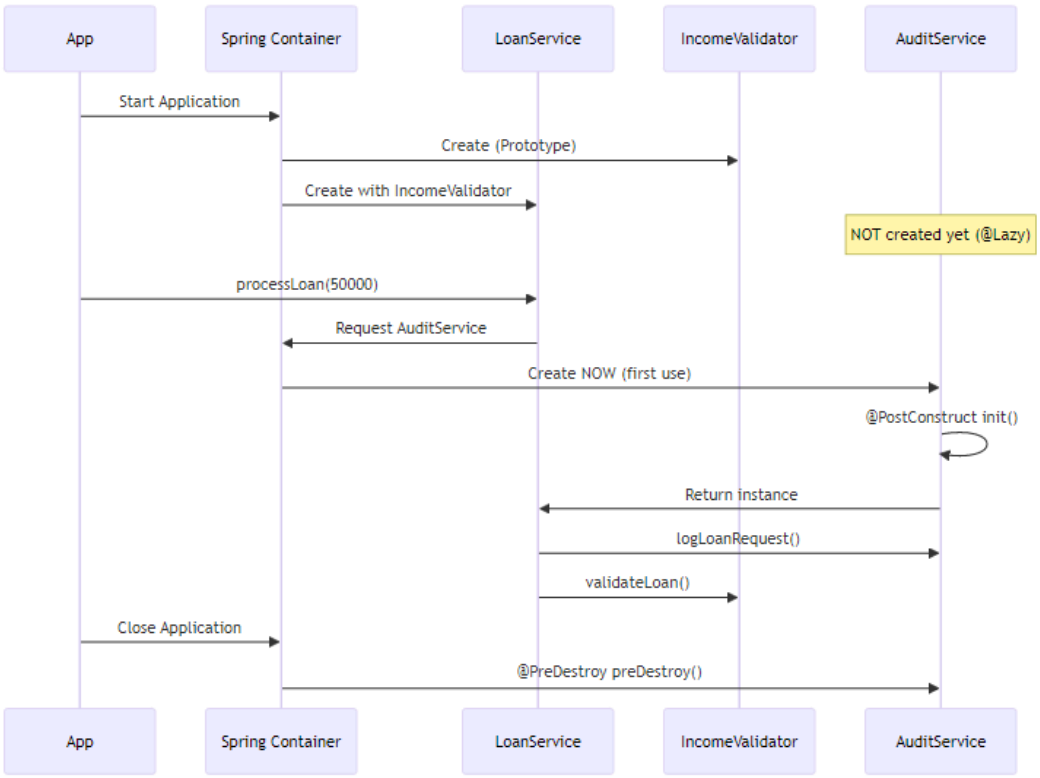
Key Points: - @Lazy → Created only when first used - @PostConstruct → Initialization logic - @PreDestroy → Cleanup logic - Injected via setter (optional dependency)

5. LoanService (Main Service)

```
@Component
public class LoanService {
    private final LoanValidator loanValidator;
    private AuditService auditService;

    // Constructor Injection with @Qualifier
    @Autowired
    public LoanService(@Qualifier("incomeValidator") LoanValidator
         loanValidator) {
        this.loanValidator = loanValidator;
        System.out.println("LoanService created with " +
         loanValidator.getClass().getSimpleName());
    }

    // Setter Injection
    @Autowired
    public void setAuditService(AuditService auditService) {
        this.auditService = auditService;
    }

    public void processLoan(double amount) {
        System.out.println("LoanService: Processing loan application");
        auditService.logLoanRequest(amount,
         loanValidator.getClass().getSimpleName());
        loanValidator.validateLoan(amount);
        System.out.println("LoanService: Loan approved");
    }
}
```

Key Points: - @Qualifier(“incomeValidator”) → Overrides @Primary - Constructor injection for required dependency - Setter injection for optional dependency - Final field for immutability

Execution Flow



Why This Design?

@Qualifier Usage: - Bank wants to use IncomeValidator specifically - Overrides the @Primary CreditScoreValidator - Explicit selection for business requirements

Prototype Scope for IncomeValidator: - Each loan application gets fresh validator - Prevents state pollution between requests - Useful if validator maintains state

@Lazy for AuditService: - Audit might not be needed in all scenarios - Saves startup time - Created only when loan processing starts

Setter Injection for AuditService: - Audit is optional feature - Can be null without breaking LoanService - Allows runtime configuration

Output Example

```
LoanService created with IncomeValidator
```

```
--- Using IncomeValidator (via @Qualifier) ---
LoanService: Processing loan application
AuditService initialized
AUDIT: Loan request for $50000.0 using IncomeValidator
IncomeValidator: Checking income for loan of $50000.0
LoanService: Loan approved

--- Using CreditScoreValidator (via @Primary) ---
CreditScoreValidator: Checking credit score for loan of $75000.0

Application closed
AuditService destroyed
```

---

4. CASE STUDY 2: FOOD DELIVERY SYSTEM

📌 Business Problem

An online food delivery platform needs a notification system that can send order updates via: - **Email** (default, most reliable) - **SMS** (optional, for urgent notifications)

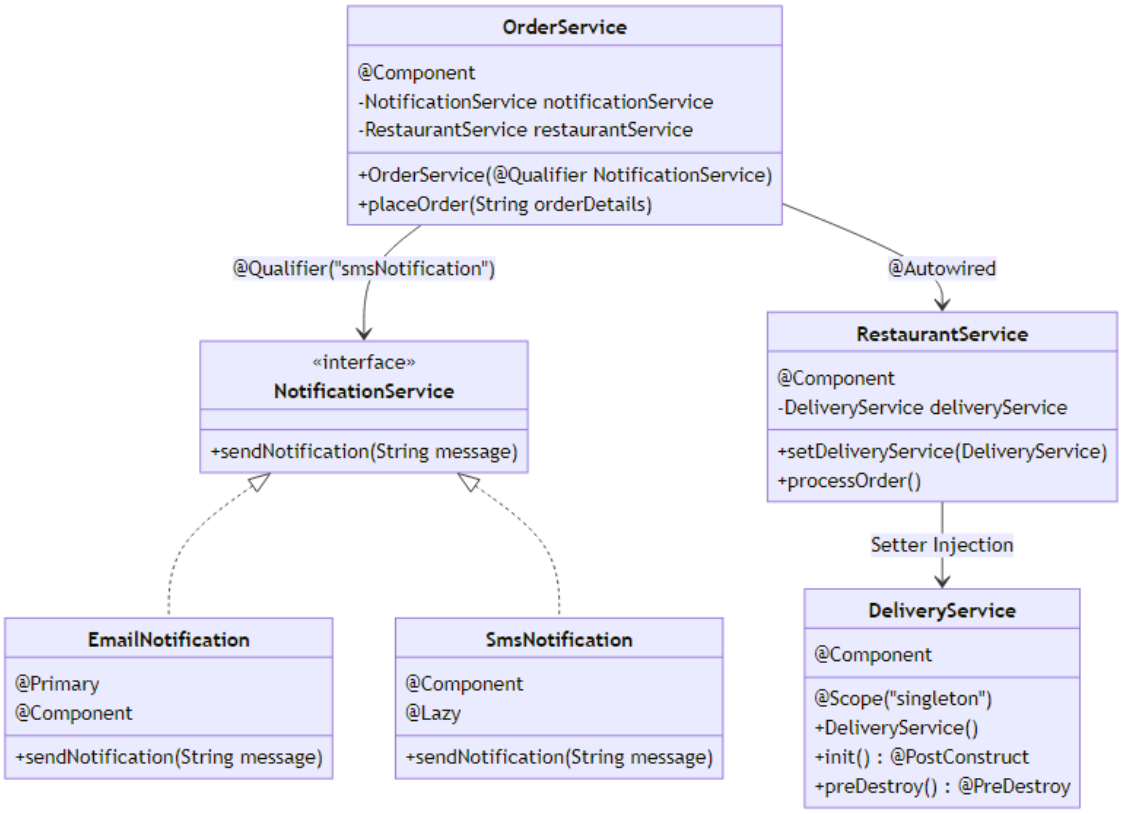
The system must handle order processing, restaurant coordination, and delivery tracking with proper lifecycle management.

🎯 Spring Concepts Demonstrated

1. **@Primary** - EmailNotification as default notification method
2. **@Qualifier** - OrderService explicitly uses SMS
3. **@Lazy** - SmsNotification loaded only when needed
4. **Singleton Scope** - DeliveryService shared across application
5. **@PostConstruct/@PreDestroy** - DeliveryService lifecycle
6. **Constructor Injection** - NotificationService in OrderService
7. **Setter Injection** - DeliveryService in RestaurantService

8. **Multiple Bean Resolution** - Different services use different notifications

📊 Architecture Diagram



🔍 Component Breakdown

1. NotificationService Interface

```
public interface NotificationService {
    void sendNotification(String message);
}
```

Purpose: Contract for different notification channels

2. EmailNotification (@Primary)

```
@Primary
@Component
public class EmailNotification implements NotificationService {
```

```
    {
        System.out.println("Non-Static Block : Email Service is launching
         !!");
    }

    @Override
    public void sendNotification(String message) {
        System.out.println("📧 EMAIL: " + message);
    }
}
```

Key Points: - @Primary → Default notification method - Non-static initialization block - Most reliable notification channel - Used when no @Qualifier specified

3. SmsNotification (@Lazy)

```
@Component
@Lazy
public class SmsNotification implements NotificationService {
    static {
        System.out.println("Static Block : SMS Service is launching !!");
    }

    @Override
    public void sendNotification(String message) {
        System.out.println("📱 SMS: " + message);
    }
}
```

Key Points: - @Lazy → Created only when requested - Static initialization block - Expensive SMS gateway connection - Used with @Qualifier for explicit selection

4. DeliveryService (Singleton with Lifecycle)

```
@Component
@Scope("singleton")
public class DeliveryService {
    public DeliveryService() {
        System.out.println("-- Delivery Service Activated --");
    }

    @PostConstruct
    public void init() {
        System.out.println("Init Method: Delivery Service Ready");
```

```
    }

    @PreDestroy
    public void preDestroy() {
        System.out.println("Delivery Service Closed");
    }
}
```

Key Points: - Singleton scope → One instance for entire application - @Post-Construct → Initialize delivery tracking - @PreDestroy → Cleanup delivery re-sources - Shared across all orders

5. RestaurantService (Setter Injection)

```
@Component
public class RestaurantService {
    private DeliveryService deliveryService;

    @Autowired
    public void setDeliveryService(DeliveryService deliveryService) {
        this.deliveryService = deliveryService;
        System.out.println("-- Setter Injection: DeliveryService injected
         into RestaurantService --");
    }

    public void processOrder() {
        System.out.println("Restaurant is processing the order...");
    }
}
```

Key Points: - Setter injection for DeliveryService - Optional dependency pattern - Allows runtime configuration

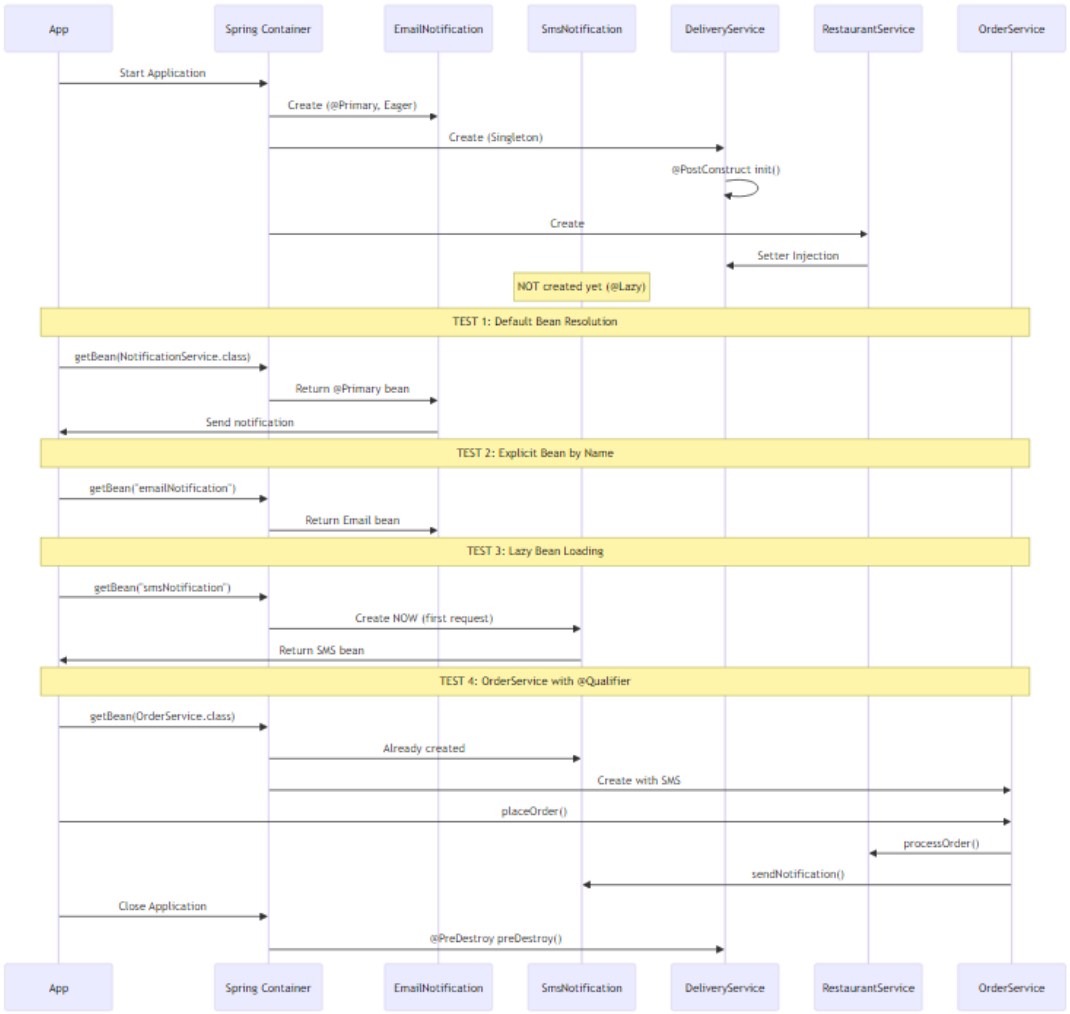
6. OrderService (Constructor Injection with @Qualifier)

```
@Component
public class OrderService {
    private final NotificationService notificationService;

    @Autowired
    private RestaurantService restaurantService;

    public OrderService(@Qualifier("smsNotification") NotificationService
         notificationService) {
```

```
        this.notificationService = notificationService;
        System.out.println("-- Constructor Injection: NotificationService
         injected into OrderService --");
    }

    public void placeOrder(String orderDetails) {
        System.out.println("\n=== Placing Order ===");
        restaurantService.processOrder();
        notificationService.sendNotification("Order placed: " +
         orderDetails);
        System.out.println("=== Order Completed ===\n");
    }
}
```

Key Points: - @Qualifier(“smsNotification”) → Overrides @Primary - Constructor injection for required dependency - Field injection for RestaurantService - Immutable notificationService with final

🎼 Execution Flow



🎯 Why This Design?

@Primary for EmailNotification: - Email is the most reliable notification method - Default choice for most services - Fallback when no specific channel requested

@Qualifier in OrderService: - Orders need immediate SMS notification - Overrides @Primary for urgent updates - Business requirement for real-time alerts

@Lazy for SmsNotification: - SMS gateway connection is expensive - Not all services need SMS - Created only when OrderService is used

Singleton for DeliveryService: - Shared delivery tracking across all orders - Single point of coordination - Resource efficiency

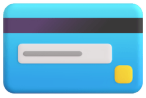
Lifecycle Management: - @PostConstruct → Initialize delivery tracking system - @PreDestroy → Close delivery connections, save state

📈 Output Example

```
--- Online Food Delivery System ----
Non-Static Block : Email Service is launching !!
-- Delivery Service Activated --
Init Method: Delivery Service Ready
-- Setter Injection: DeliveryService injected into RestaurantService --

TEST 1: Default Bean Resolution (@Primary)
📧 EMAIL: Testing default notification (should be Email)

TEST 2: Explicit Bean Resolution by Name
📧 EMAIL: Explicitly requesting Email notification

TEST 3: @Lazy Bean Resolution
Requesting SMS Notification (Lazy-loaded)...
Static Block : SMS Service is launching !!
📱 SMS: SMS notification loaded lazily

TEST 4: OrderService with @Qualifier Override
OrderService uses @Qualifier("smsNotification") to override @Primary
-- Constructor Injection: NotificationService injected into OrderService --

=== Placing Order ===
Restaurant is processing the order...
📱 SMS: Order placed: Pizza Margherita x2, Coke x1
=== Order Completed ===

Closing Application Context...
Delivery Service Closed

Application Shutdown Complete!
```

---

5. CASE STUDY 3: PAYMENT PROCESSING



📌 Business Problem

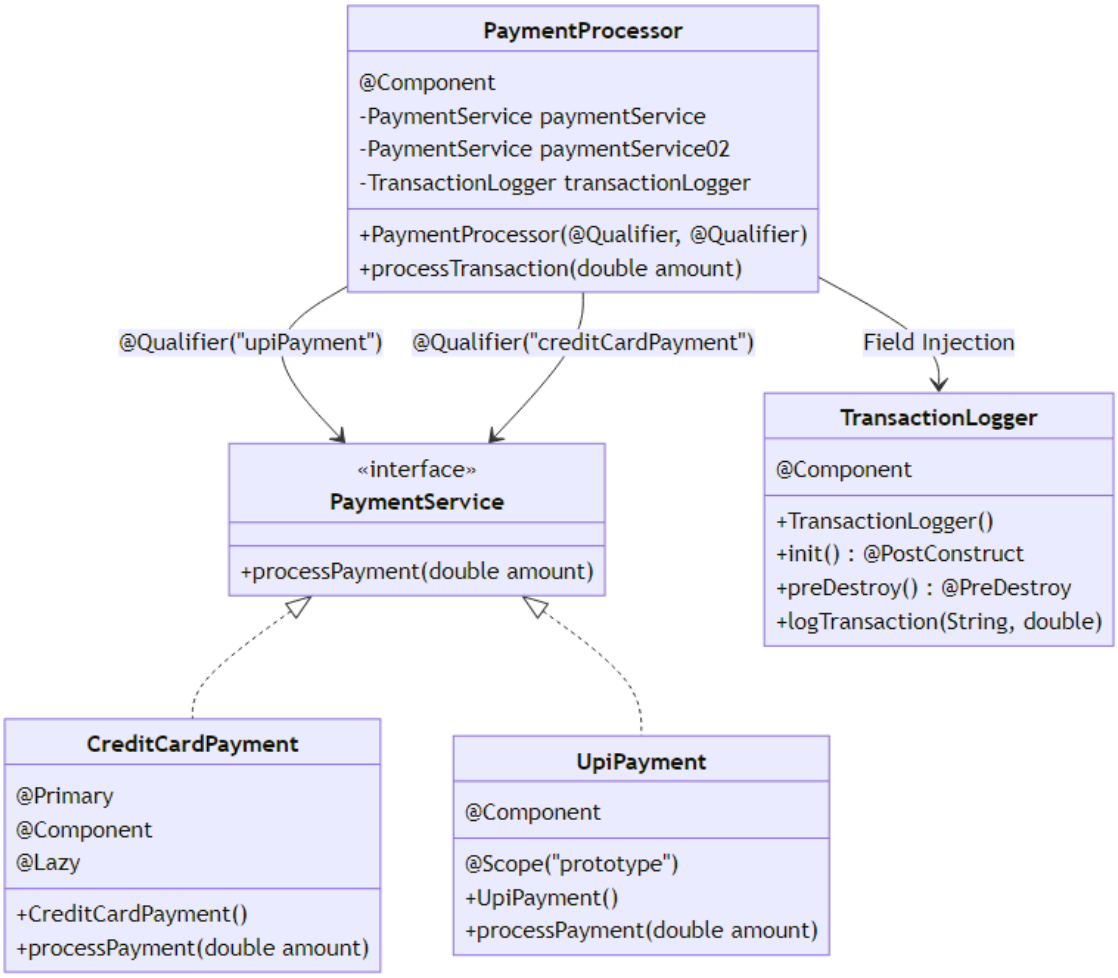
A payment gateway needs to support multiple payment methods: - **Credit Card** (default, most common) - **UPI** (popular in India, fast processing)

The system must log all transactions, support multiple payment methods simultaneously, and manage resources efficiently.

🎯 Spring Concepts Demonstrated

1. **@Primary + @Lazy** - CreditCardPayment as default but lazy-loaded
2. **Prototype Scope** - New UpiPayment instance per transaction
3. **Multiple @Qualifier** - PaymentProcessor uses both payment methods
4. **@PostConstruct/@PreDestroy** - TransactionLogger lifecycle
5. **Constructor Injection** - Multiple dependencies with @Qualifier
6. **Field Injection** - TransactionLogger injected via field
7. **Combining Annotations** - @Primary, @Lazy, @Qualifier together

🏛 Architecture Diagram



🔍 Component Breakdown

1. PaymentService Interface

```java
public interface PaymentService {
    void processPayment(double amount);
}
```

Purpose: Contract for different payment methods

2. CreditCardPayment (@Primary + @Lazy)

```java
@Primary
```

```
@Component
@Lazy
public class CreditCardPayment implements PaymentService {
    public CreditCardPayment() {
        System.out.println("CreditCardPayment bean created (Lazy)");
    }

    @Override
    public void processPayment(double amount) {
        System.out.println("Processing Credit Card payment of $" + amount);
    }
}
```

Key Points: - @Primary → Default payment method - @Lazy → Created only when used - Singleton scope (default) - Expensive credit card gateway initialization

3. UpiPayment (Prototype)

```
@Component
@Scope("prototype")
public class UpiPayment implements PaymentService {
    public UpiPayment() {
        System.out.println("UpiPayment bean created (Prototype)");
    }

    @Override
    public void processPayment(double amount) {
        System.out.println("Processing UPI payment of " + amount);
    }
}
```

Key Points: - Prototype scope → New instance per request - Fresh instance for each transaction - Prevents transaction state mixing - Must use @Qualifier to inject

4. TransactionLogger (Lifecycle Management)

```
@Component
public class TransactionLogger {
    public TransactionLogger() {
        System.out.println("TransactionLogger bean created");
    }
```

```
    @PostConstruct
    public void init() {
        System.out.println("Logger initialized");
    }

    @PreDestroy
    public void preDestroy() {
        System.out.println("Logger destroyed");
    }

    public void logTransaction(String paymentType, double amount) {
        System.out.println("Transaction logged: " + paymentType + " - " +
         amount);
    }
}
```

Key Points: - Singleton scope → One logger for all transactions - @PostConstruct → Open log file, initialize database connection - @PreDestroy → Flush logs, close connections - Shared resource management

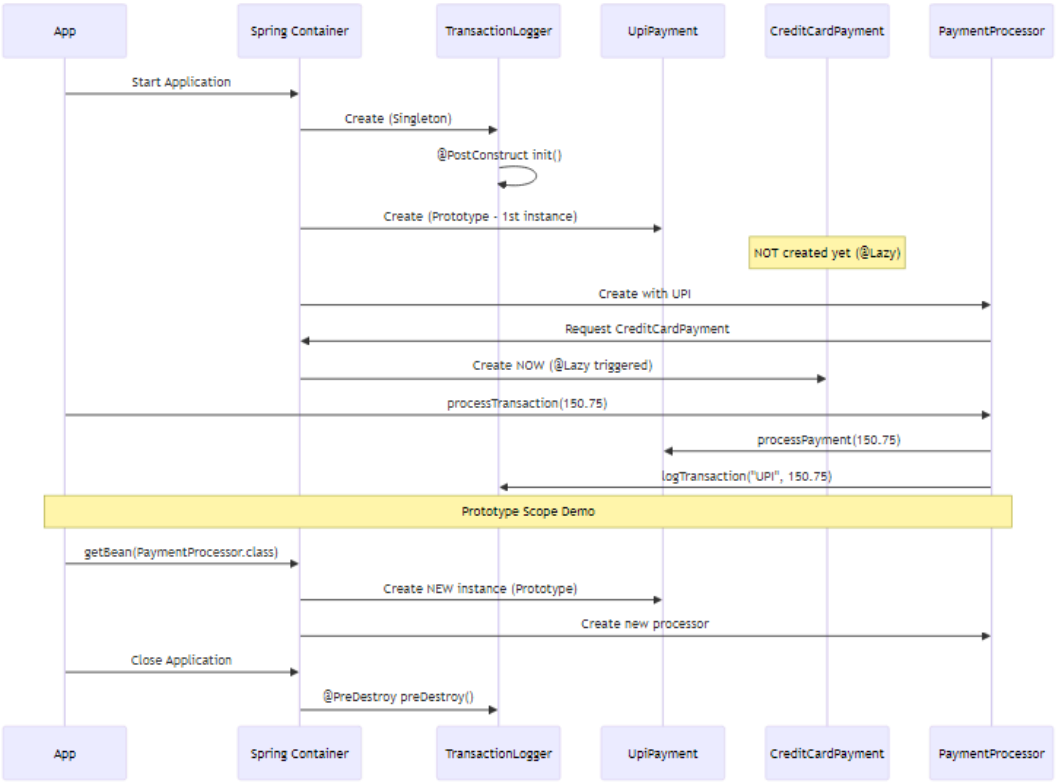
5. PaymentProcessor (Multiple @Qualifier)

```
@Component
public class PaymentProcessor {
    private final PaymentService paymentService;
    private final PaymentService paymentService02;

    @Autowired
    private TransactionLogger transactionLogger;

    // Constructor injection with multiple @Qualifier
    public PaymentProcessor(
            @Qualifier("upiPayment") PaymentService paymentService,
            @Qualifier("creditCardPayment") PaymentService
         paymentService02) {
        this.paymentService02 = paymentService02;
        this.paymentService = paymentService;
        System.out.println("PaymentProcessor bean created");
    }

    public void processTransaction(double amount) {
        System.out.println("\n--- Processing ---");
        paymentService.processPayment(amount);
        transactionLogger.logTransaction("UPI", amount);
        System.out.println("--- Transaction Complete ---\n");
```

```
    }
}
```

Key Points: - Two PaymentService dependencies with different @Qualifier - Constructor injection for payment services - Field injection for TransactionLogger - Can switch between payment methods dynamically

📊 Execution Flow



🎯 Why This Design?

@Primary + @Lazy for CreditCardPayment: - Credit card is most common payment method - But expensive gateway initialization - Created only when actually used - Combines default selection with performance optimization

Prototype for UpiPayment: - Each transaction needs fresh payment instance - Prevents transaction state leakage - Stateful payment processing - New instance = clean state

Multiple @Qualifier in PaymentProcessor: - System supports multiple payment methods - Can process different payment types - Flexibility to switch at runtime - Demonstrates complex dependency resolution

Singleton for TransactionLogger: - Centralized logging for all transactions - Shared resource (log file, database) - Lifecycle management for cleanup - Resource efficiency

📈 Output Example

```
=== Starting Payment Processing System ===

TransactionLogger bean created
Logger initialized
UpiPayment bean created (Prototype)
CreditCardPayment bean created (Lazy)
PaymentProcessor bean created

--- Processing ---
Processing UPI payment of 150.75
Transaction logged: UPI - 150.75
--- Transaction Complete ---

=== Demonstrating Prototype Scope ===
UpiPayment bean created (Prototype)
PaymentProcessor bean created

--- Processing ---
Processing UPI payment of 200.0
Transaction logged: UPI - 200.0
--- Transaction Complete ---

=== Shutting Down ===
Logger destroyed

=== Application Terminated ===
```

---

6. CORE CONCEPTS DEEP DIVE

📌 Spring Dependency Injection Fundamentals

What is Dependency Injection?

Dependency Injection is a design pattern where objects receive their dependencies from external sources rather than creating them internally. Spring IoC Container manages this process automatically.

Traditional Approach (Without DI):

```
public class OrderService {
    private NotificationService notificationService;

    public OrderService() {
        // ❌ Tight coupling - creates dependency internally
        this.notificationService = new EmailNotification();
    }
}
```

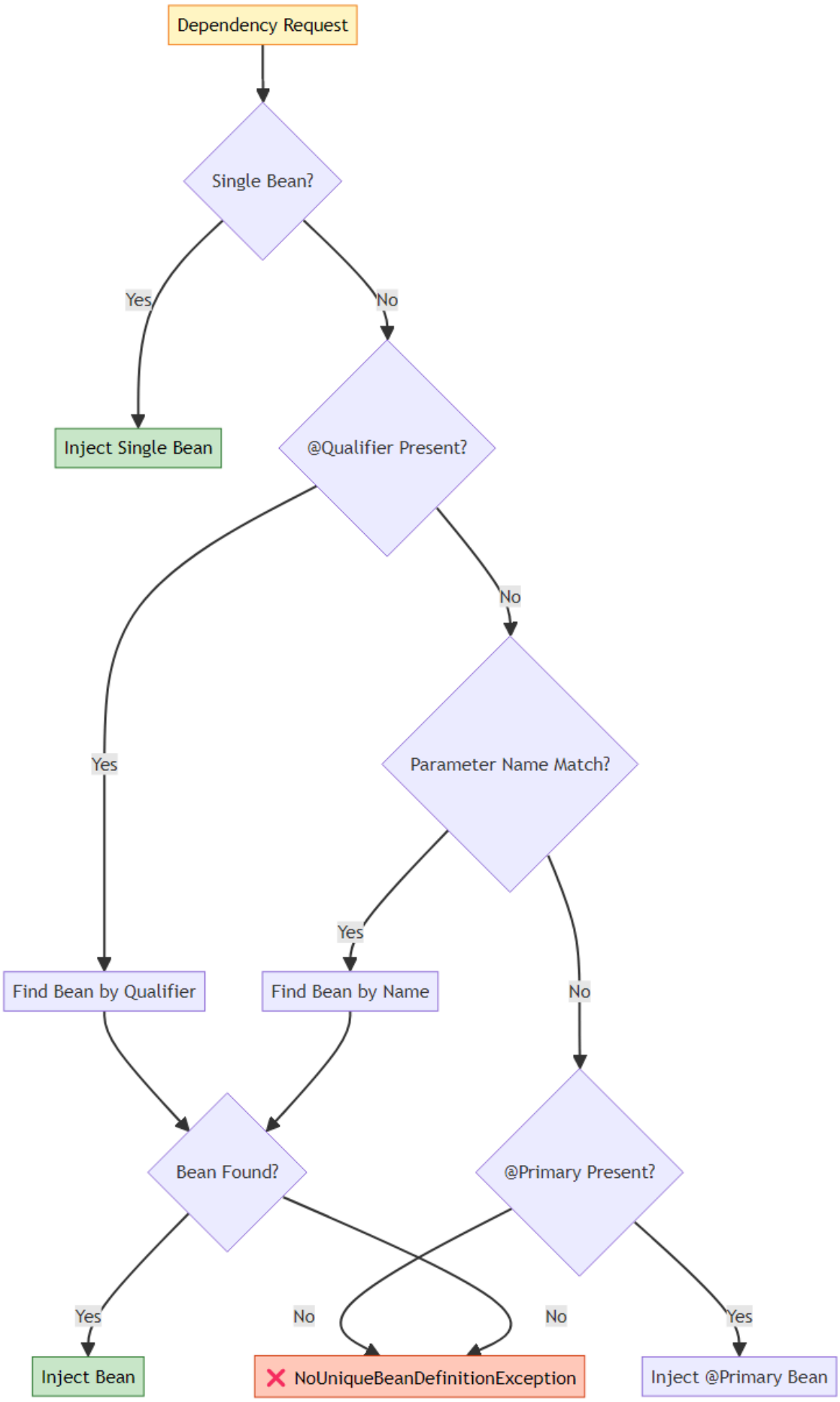
Problems: - Tight coupling between classes - Hard to test (can't mock dependencies) - Difficult to change implementation - Violates Single Responsibility Principle

Spring DI Approach:

```
@Component
public class OrderService {
    private final NotificationService notificationService;

    // ✅ Loose coupling - dependency injected
    @Autowired
    public OrderService(NotificationService notificationService) {
        this.notificationService = notificationService;
    }
}
```

Benefits: - Loose coupling - Easy to test (inject mocks) - Flexible implementation switching - Follows SOLID principles

🎯 How Spring Resolves Dependencies


Dependency Request
Single Bean?
Yes
No
Inject Single Bean
@Qualifier Present?
Yes
No
Parameter Name Match?
Yes
No
Find Bean by Qualifier
Find Bean by Name
Bean Found?
@Primary Present?
Yes
No
No
Yes
Inject Bean
NoUniqueBeanDefinitionException
Inject @Primary Bean

📊 Resolution Priority Table

Priority	Mechanism	Example	Overrides
1 (Highest)	@Qualifier	`@Qualifier("smsNotification")`	Everything
2	Parameter Name	`NotificationService emailNotification`	@Primary, Single
3	@Primary	`@Primary @Component`	Single Bean
4 (Lowest)	Single Bean	Only one implementation	Nothing

---

7. @PRIMARY VS @QUALIFIER

📌 What is @Primary?

@Primary marks a bean as the default choice when multiple beans of the same type exist. It's a fallback mechanism when no explicit selection is made.

When Spring Uses @Primary: - No @Qualifier specified - Parameter name doesn't match any bean - Multiple beans of same type exist

Example from FoodDelivery:

```
@Primary
@Component
public class EmailNotification implements NotificationService {
    // This is the DEFAULT notification method
}

@Component
public class SmsNotification implements NotificationService {
    // Alternative notification method
}

// Usage
```

```java
@Component
public class SomeService {
    private final NotificationService service;

    // Gets EmailNotification (marked with @Primary)
    public SomeService(NotificationService service) {
        this.service = service;
    }
}
```

📌 What is @Qualifier?

@Qualifier explicitly selects a specific bean by name. It overrides @Primary and provides precise control over dependency resolution.

When to Use @Qualifier: - Need specific implementation - Override @Primary selection - Multiple beans of same type - Explicit business requirement

Example from BankLoanApproval:

```java
@Component
@Primary
public class CreditScoreValidator implements LoanValidator {
    // Default validator
}

@Component
public class IncomeValidator implements LoanValidator {
    // Alternative validator
}

// Usage
@Component
public class LoanService {
    private final LoanValidator validator;

    // Gets IncomeValidator (overrides @Primary)
    public LoanService(@Qualifier("incomeValidator") LoanValidator
         validator) {
        this.validator = validator;
    }
}
```

🎯 @Primary vs @Qualifier Comparison

Aspect	@Primary	@Qualifier
Purpose	Default selection	Explicit selection
Priority	Lower (fallback)	Higher (overrides)
Usage	On bean definition	On injection point
Count	Only ONE per type	Multiple allowed
Flexibility	Less flexible	More flexible
Use Case	Common default	Specific requirement
Overrides	Single bean only	Everything

📊 Resolution Scenarios

Scenario 1: No @Qualifier, @Primary Exists

```
@Primary
@Component
class EmailNotification implements NotificationService { }

@Component
class SmsNotification implements NotificationService { }

@Component
class MyService {
    // Gets EmailNotification (@Primary)
    public MyService(NotificationService service) { }
}
```

Scenario 2: @Qualifier Overrides @Primary

```
@Primary
@Component
class EmailNotification implements NotificationService { }

@Component
class SmsNotification implements NotificationService { }

@Component
class MyService {
    // Gets SmsNotification (@Qualifier overrides @Primary)
```

```
    public MyService(@Qualifier("smsNotification") NotificationService
        service) { }
}
```

Scenario 3: Parameter Name Matching

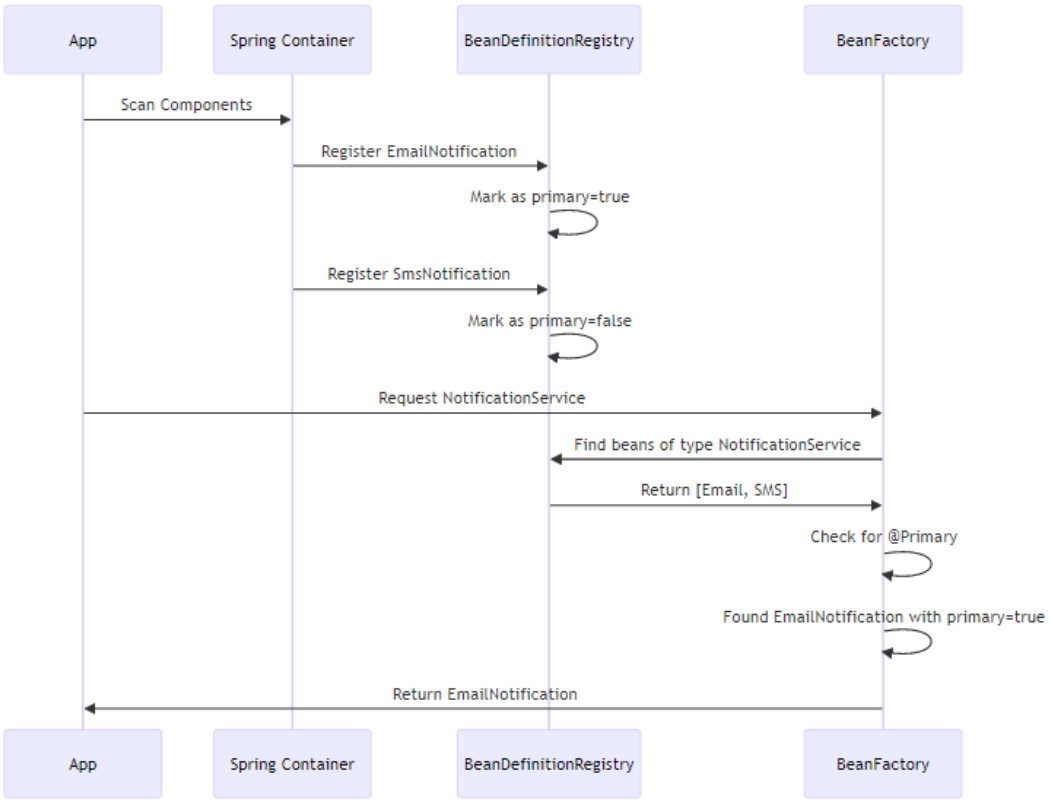
```
@Primary
@Component
class EmailNotification implements NotificationService { }

@Component
class SmsNotification implements NotificationService { }

@Component
class MyService {
    // Gets SmsNotification (parameter name matches bean name)
    public MyService(NotificationService smsNotification) { }
}
```

Scenario 4: Multiple @Primary (ERROR)

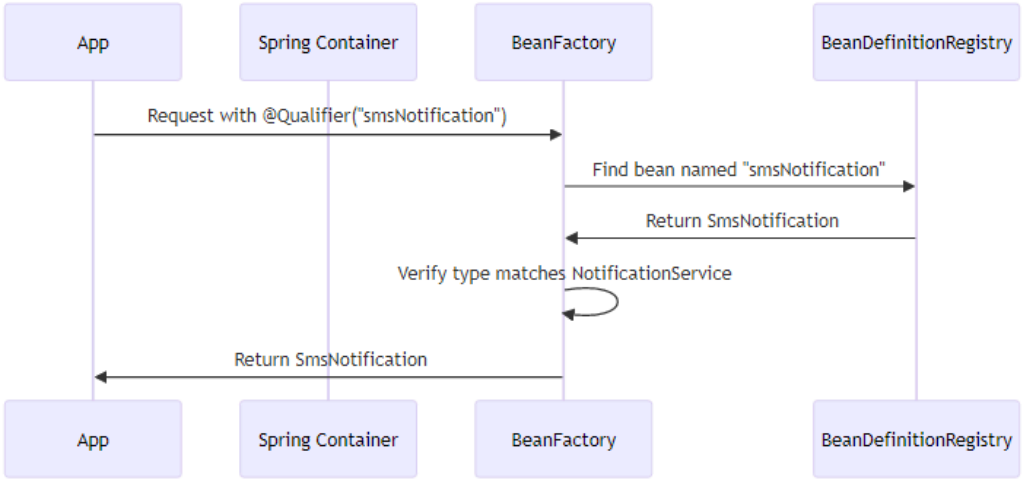
```
@Primary
@Component
class EmailNotification implements NotificationService { }

@Primary  // ❌ ERROR: Only ONE @Primary allowed per type
@Component
class SmsNotification implements NotificationService { }
```

🔍 Internal Working

How Spring Processes @Primary:



How Spring Processes @Qualifier:



🎯 Best Practices

Use @Primary When: - ✅ One implementation is clearly the default - ✅ Most services should use this implementation - ✅ Fallback behavior is desired - ✅ Simplify configuration for common case

Use @Qualifier When: - ✅ Need specific implementation - ✅ Business logic requires particular bean - ✅ Override default behavior - ✅ Multiple beans needed in same class

Avoid: - ❌ Multiple @Primary for same type - ❌ @Qualifier without clear reason - ❌ Overusing @Qualifier (consider separate interfaces) - ❌ Mixing @Primary with unclear naming

---

8. @LAZY INITIALIZATION

📌 What is @Lazy?

@Lazy tells Spring to delay bean creation until it's first requested, rather than creating it at application startup. This is a performance optimization technique.

Default Behavior (Eager):

```
@Component
public class EmailService {
    public EmailService() {
        System.out.println("EmailService created at startup");
    }
}
// Output at startup: "EmailService created at startup"
```

With @Lazy:

```
@Component
@Lazy
```

```
public class SmsService {
    public SmsService() {
        System.out.println("SmsService created on first use");
    }
}
// No output at startup
// Output when first used: "SmsService created on first use"
```

🎯 Why @Lazy Exists

Problem: Slow Application Startup

Imagine an application with 100 beans, each taking 1 second to initialize: - Without @Lazy: 100 seconds startup time - With @Lazy (50% lazy): 50 seconds startup time - User waits less, application feels faster

Real-World Example from Case Studies:

AuditService in BankLoanApproval:

```
@Component
@Lazy
public class AuditService {
    @PostConstruct
    public void init() {
        // Expensive: Connect to audit database
        // Load audit configuration
        // Initialize audit logger
        System.out.println("AuditService initialized");
    }
}
```

Why Lazy? - Audit might not be needed immediately - Expensive database connection - Not all loan operations require audit - Faster application startup

SmsNotification in FoodDelivery:

```
@Component
@Lazy
public class SmsNotification implements NotificationService {
    static {
        // Expensive: Connect to SMS gateway
        // Load SMS templates
        // Initialize SMS client
```

```
        System.out.println("SMS Service is launching !!");
    }
}
```

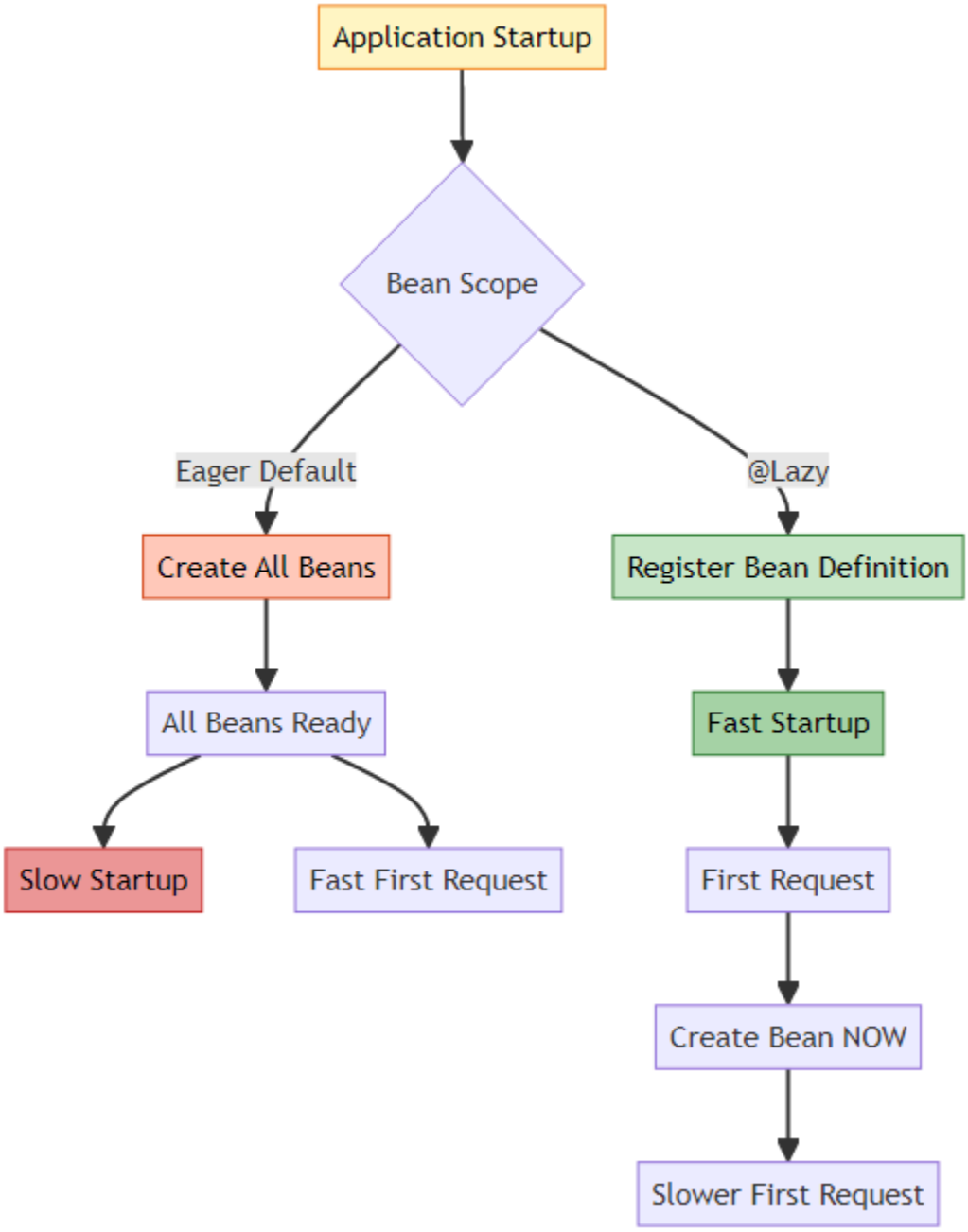
Why Lazy? - SMS gateway connection is expensive - Not all orders need SMS notification - Email is default (@Primary) - SMS used only for urgent orders

CreditCardPayment in PaymentProcessing:

```
@Primary
@Component
@Lazy
public class CreditCardPayment implements PaymentService {
    public CreditCardPayment() {
        // Expensive: Connect to payment gateway
        // Load encryption keys
        // Initialize payment processor
        System.out.println("CreditCardPayment bean created (Lazy)");
    }
}
```

Why Lazy? - Payment gateway initialization is expensive - Not all application features need payment - Created only when payment is processed - Combines @Primary (default) with @Lazy (performance)

📊 Eager vs Lazy Comparison

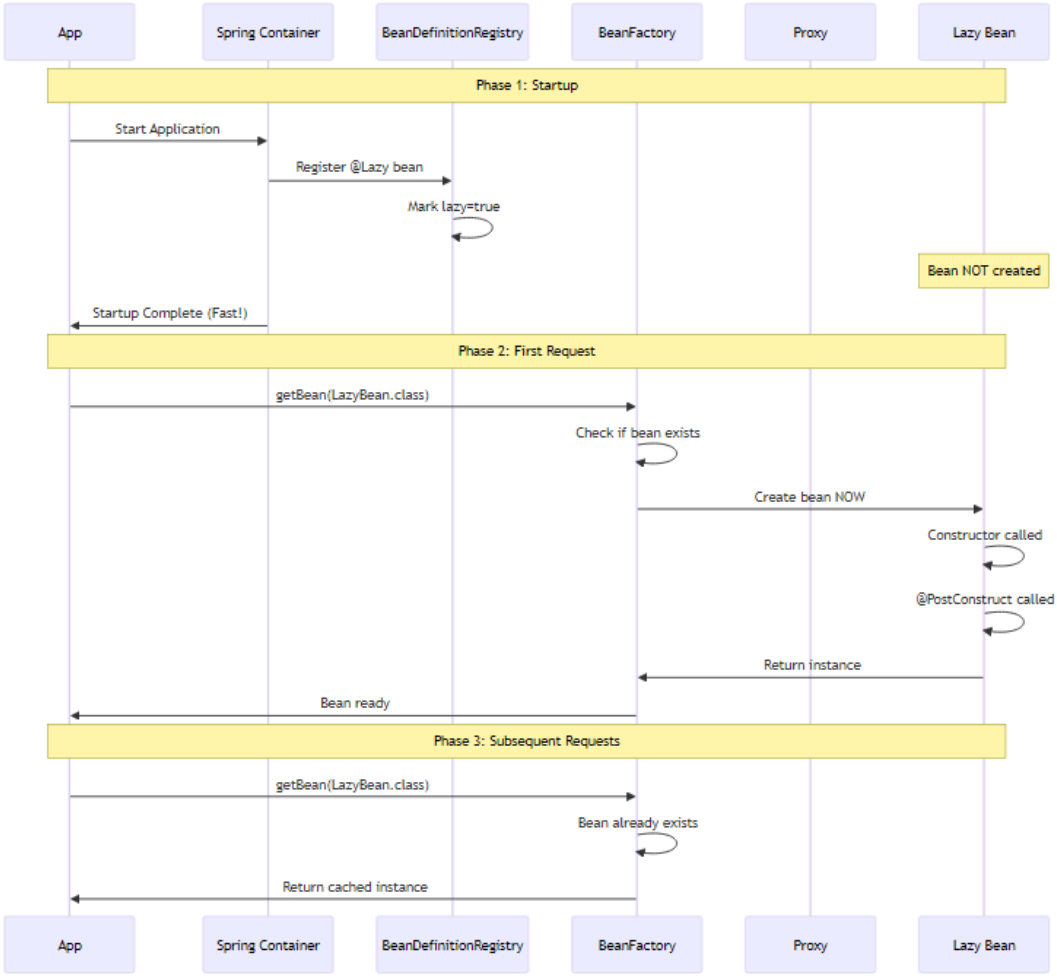


Aspect	Eager (Default)	@Lazy
Creation Time	Application startup	First use
Startup Speed	❌ Slower	✅ Faster

Aspect	Eager (Default)	@Lazy
First Request	✅ Fast	❌ Slower
Memory Usage	❌ Higher (all beans)	✅ Lower (only used)
Error Detection	✅ Immediate	❌ Delayed
Use Case	Always-used beans	Rarely-used beans

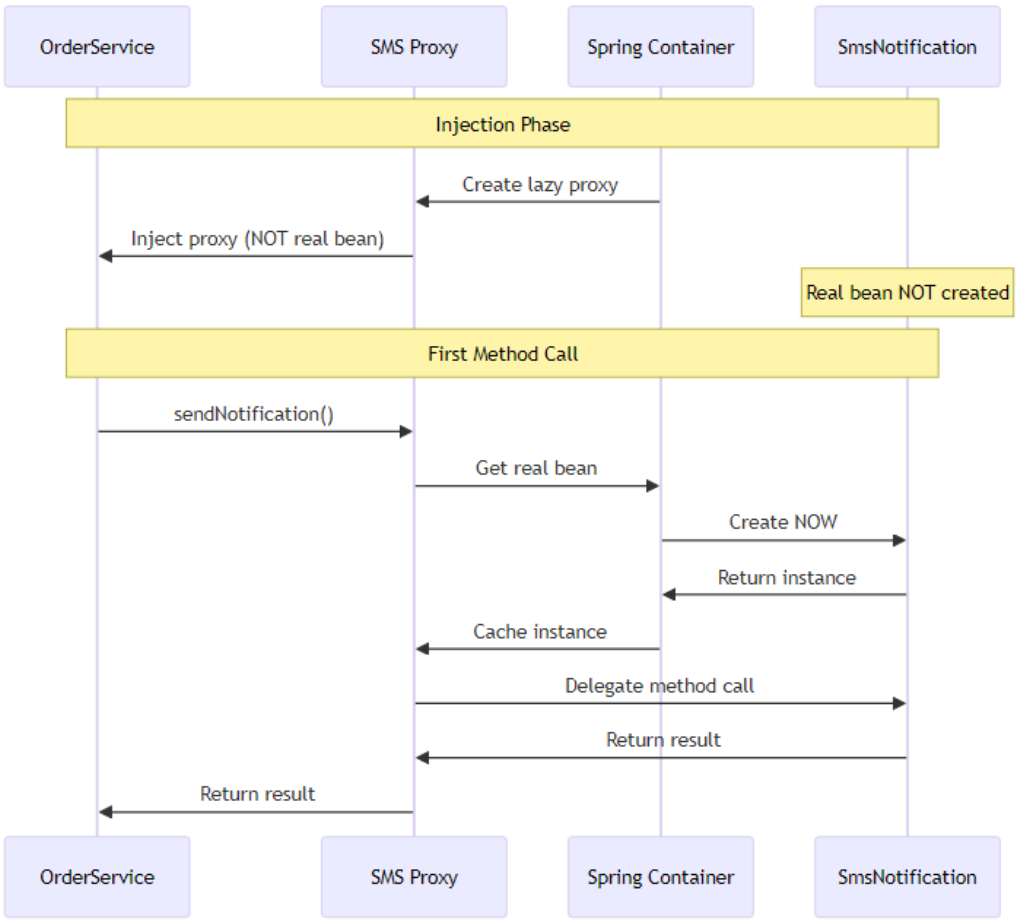
🔍 Internal Working

Lazy Bean Creation Process:



Lazy with Dependency Injection (Proxy Pattern):

When @Lazy bean is injected into another bean, Spring creates a proxy:



🎯 When to Use @Lazy

Use @Lazy When: - ✅ Bean initialization is expensive (database, network) - ✅ Bean is rarely used - ✅ Want faster application startup - ✅ Optional features that may not be needed - ✅ Breaking circular dependencies

Avoid @Lazy When: - ❌ Bean is always used - ❌ Want fail-fast behavior - ❌ Initialization is quick - ❌ Need predictable performance

📈 Performance Impact

Example: E-Commerce Application

Without @Lazy:

```
Startup Time: 30 seconds
```

```
- EmailService: 2s
- SmsService: 3s
- PaymentGateway: 5s
- ReportGenerator: 8s
- DataExporter: 7s
- Other services: 5s

First Request: 10ms
```

With @Lazy (50% lazy):

```
Startup Time: 12 seconds
- EmailService: 2s (eager - always used)
- PaymentGateway: 5s (eager - critical)
- Other services: 5s

First Request: 25ms (creates lazy beans)
- SmsService: 3s (lazy - rarely used)
- ReportGenerator: 8s (lazy - admin only)
- DataExporter: 7s (lazy - admin only)

User Experience: 60% faster startup!
```

---

9. BEAN SCOPES

📌 What are Bean Scopes?

Bean Scope defines the lifecycle and visibility of a bean instance. It determines how many instances Spring creates and when they're created/destroyed.

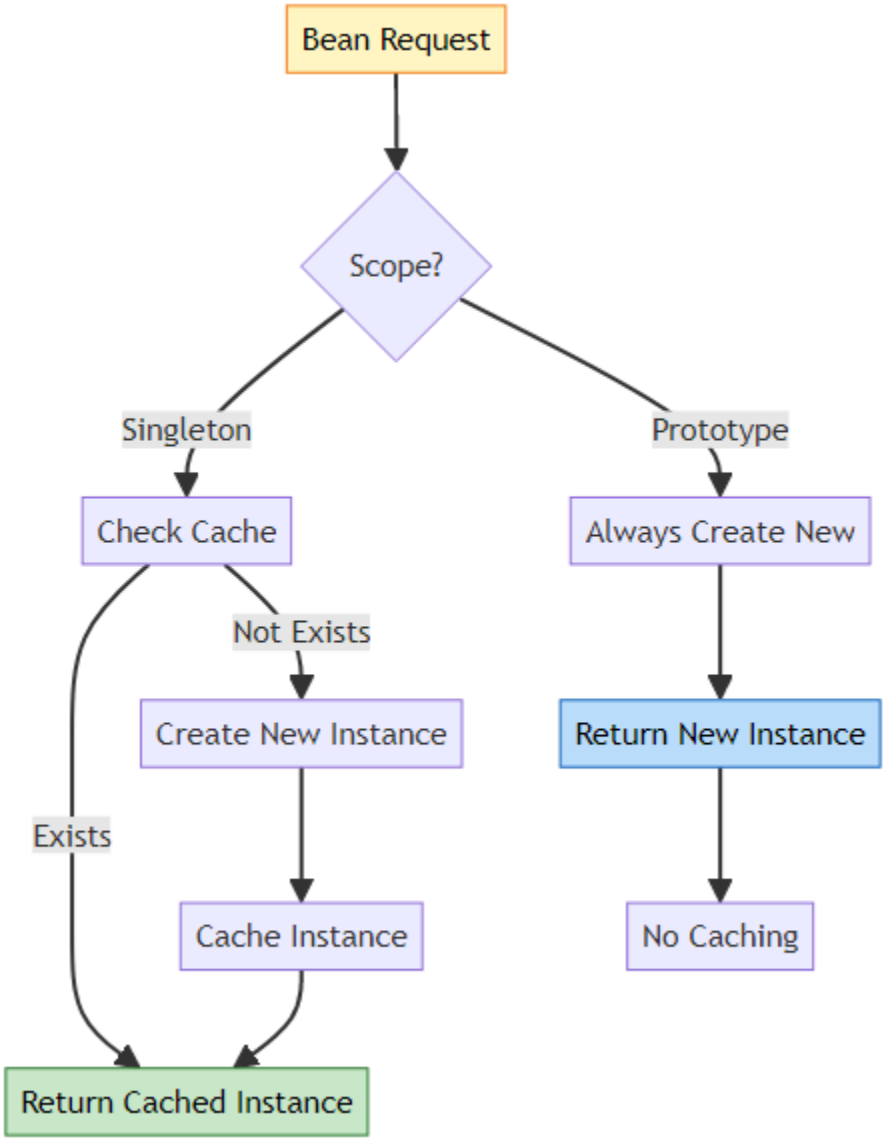
🎯 Available Scopes

Scope	Instances	Lifecycle	Use Case
Singleton	1 per container	Container lifetime	Stateless services

Scope	Instances	Lifecycle	Use Case
Prototype	New per request	Until garbage collected	Stateful objects
Request	1 per HTTP request	Request lifetime	Web - Request data
Session	1 per HTTP session	Session lifetime	Web - User session
Application	1 per ServletContext	Application lifetime	Web - Shared data
WebSocket	1 per WebSocket	WebSocket lifetime	WebSocket connections

Note: Request, Session, Application, and WebSocket scopes are only available in web applications.

🎹 Singleton vs Prototype (Our Case Studies)



🔍 Singleton Scope (Default)

Definition: Spring creates only ONE instance per container. All requests get the same instance.

Example from FoodDelivery:

```
@Component
@Scope("singleton")  // Default, can be omitted
```

```
public class DeliveryService {
    public DeliveryService() {
        System.out.println("-- Delivery Service Activated --");
    }
}
```

Behavior:

```
DeliveryService service1 = context.getBean(DeliveryService.class);
DeliveryService service2 = context.getBean(DeliveryService.class);

System.out.println(service1 == service2);  // true (same instance)
```

Output:

```
-- Delivery Service Activated --  (created once)
true
```

When to Use Singleton: - ✅ Stateless services - ✅ Shared resources (database connections, caches) - ✅ Thread-safe beans - ✅ Configuration beans - ✅ Most Spring beans

Example: TransactionLogger (PaymentProcessing)

```
@Component  // Singleton by default
public class TransactionLogger {
    // Shared logger for all transactions
    // One instance serves entire application
}
```

Why Singleton? - All transactions logged to same file - Shared resource management - Memory efficient - Thread-safe logging

🔍 Prototype Scope

Definition: Spring creates a NEW instance every time the bean is requested.

Example from BankLoanApproval:

```
@Component
@Scope("prototype")
public class IncomeValidator implements LoanValidator {
    @Override
    public void validateLoan(double amount) {
```

```
        System.out.println("IncomeValidator: Checking income for loan of $"
         + amount);
    }
}
```

Behavior:

```
IncomeValidator validator1 = context.getBean(IncomeValidator.class);
IncomeValidator validator2 = context.getBean(IncomeValidator.class);

System.out.println(validator1 == validator2);  // false (different
         instances)
```

Output:

```
IncomeValidator created  (first request)
IncomeValidator created  (second request)
false
```

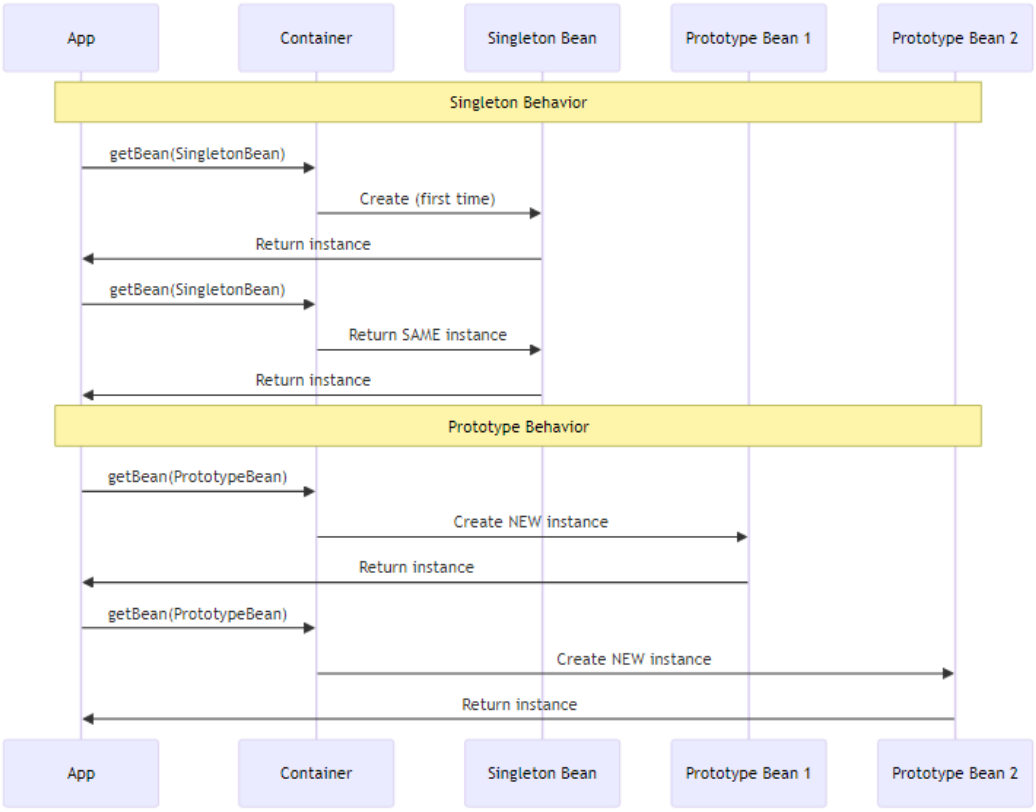
When to Use Prototype: - ✅ Stateful objects - ✅ Per-request processing - ✅ Mutable beans - ✅ Thread-unsafe beans - ✅ Temporary objects

Example: UpiPayment (PaymentProcessing)

```
@Component
@Scope("prototype")
public class UpiPayment implements PaymentService {
    // New instance per transaction
    // Prevents transaction state mixing
}
```

Why Prototype? - Each transaction needs fresh payment instance - Transaction state isolation - Prevents data leakage between transactions - Clean state for each payment

📊 Singleton vs Prototype Comparison



Aspect	Singleton	Prototype
Instances	1 per container	New per request
Creation	At startup (eager) or first use (lazy)	Every getBean() call
Caching	✅ Cached	❌ Not cached
Memory	Low (one instance)	Higher (multiple instances)
Thread Safety	Must be thread-safe	Each thread gets own instance
Lifecycle	Container manages	Container creates, app manages
@PreDestroy	✅ Called	❌ NOT called
Use Case	Stateless services	Stateful objects

🎯 Scope Interaction Gotcha

Problem: Singleton with Prototype Dependency

```
@Component
@Scope("singleton")
public class OrderService {
    @Autowired
    private PaymentProcessor processor;  // Prototype

    public void processOrder() {
        processor.process();  // ALWAYS SAME INSTANCE!
    }
}
```

Issue: Singleton is created once, so it gets ONE prototype instance and keeps reusing it!

Solution 1: Method Injection

```
@Component
@Scope("singleton")
public class OrderService {
    @Autowired
    private ApplicationContext context;

    public void processOrder() {
        PaymentProcessor processor =
         context.getBean(PaymentProcessor.class);
        processor.process();  // NEW instance each time
    }
}
```

Solution 2: @Lookup

```
@Component
@Scope("singleton")
public abstract class OrderService {
    public void processOrder() {
        PaymentProcessor processor = getProcessor();
        processor.process();  // NEW instance each time
    }

    @Lookup
    protected abstract PaymentProcessor getProcessor();
```

```
}
```

📈 Performance Implications

Singleton: - ✅ Fast (cached instance) - ✅ Low memory usage - ✅ Efficient for stateless services - ⚠️ Must be thread-safe

Prototype: - ⚠️ Slower (creates new instance) - ⚠️ Higher memory usage - ✅ No thread safety concerns - ✅ Clean state per request

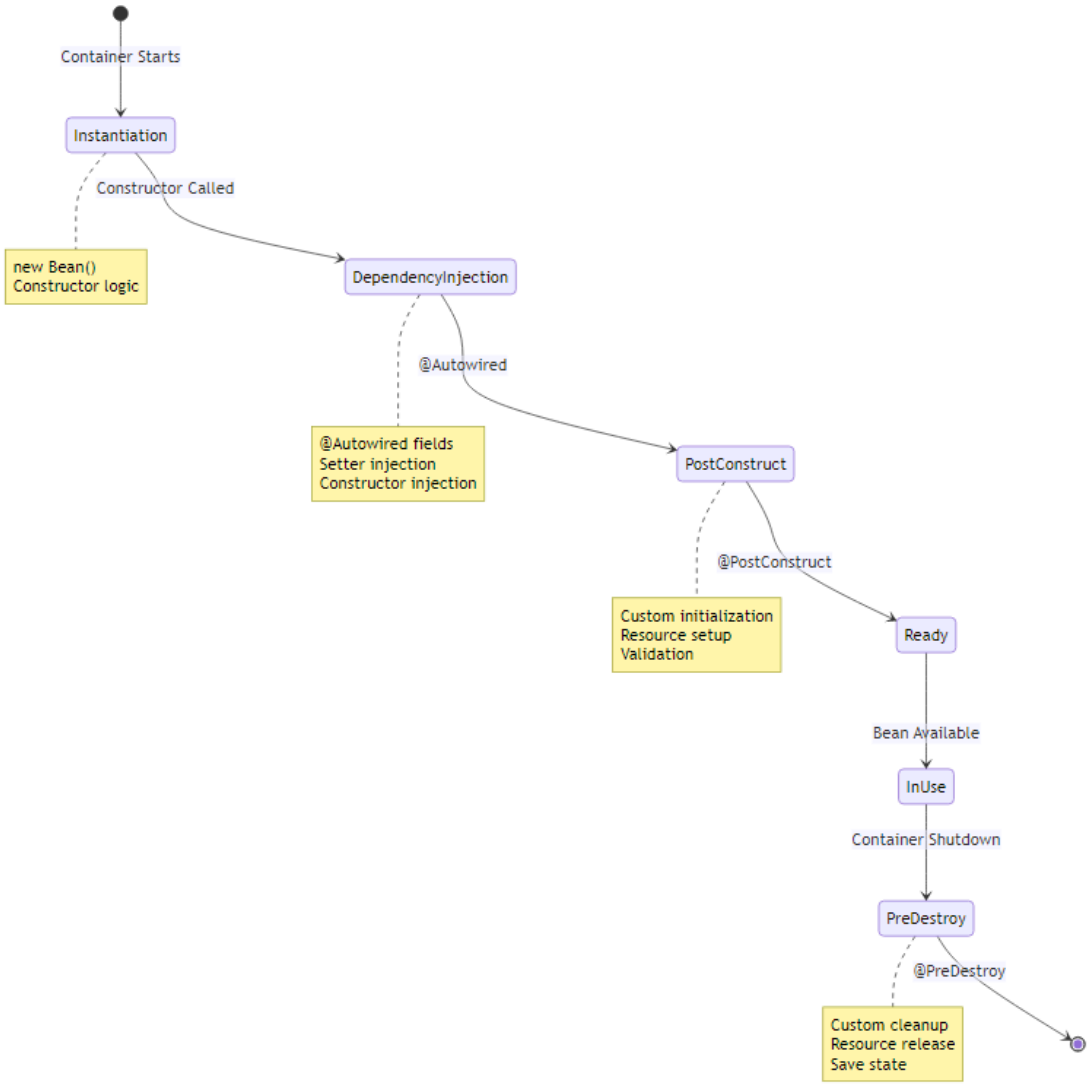
---

10. BEAN LIFECYCLE

📌 What is Bean Lifecycle?

Bean Lifecycle is the complete journey of a Spring bean from creation to destruction. Spring provides hooks at various stages for custom initialization and cleanup logic.

🎯 Complete Lifecycle Phases



📊 Lifecycle Phases Breakdown

Phase	Description	When	Example
1. Instantiation	Bean object created	Container startup	`new DeliveryService()`
2. Dependency Injection	Dependencies injected	After in-stantiation	`@Autowired` fields set
3. @PostCon-	Custom initialization	After DI	Open connec-

Phase	Description	When	Example
struct		complete	tions
4. Bean Ready	Bean available for use	After initial-ization	Application uses bean
5. @PreDestroy	Custom cleanup	Before de-struction	Close connec-tions
6. Destruction	Bean removed	Container shutdown	Memory released

🔍 @PostConstruct Deep Dive

What is @PostConstruct?

@PostConstruct marks a method to be executed AFTER dependency injection is complete. It's the recommended way to perform initialization logic.

Why @PostConstruct Exists:

Problem: Constructor Limitations

```
@Component
public class DeliveryService {
    @Autowired
    private DatabaseConnection connection;

    public DeliveryService() {
        // ❌ PROBLEM: connection is NULL here!
        connection.connect();  // NullPointerException!
    }
}
```

Solution: @PostConstruct

```
@Component
public class DeliveryService {
    @Autowired
    private DatabaseConnection connection;

    public DeliveryService() {
        System.out.println("Constructor: connection is " + connection);
        // Output: Constructor: connection is null
```

```
    }

    @PostConstruct
    public void init() {
        System.out.println("PostConstruct: connection is " + connection);
        // Output: PostConstruct: connection is DatabaseConnection@123

        // ✅ SAFE: Dependencies are injected
        connection.connect();
    }
}
```

Example from FoodDelivery:

```
@Component
@Scope("singleton")
public class DeliveryService {
    public DeliveryService() {
        System.out.println("-- Delivery Service Activated --");
    }

    @PostConstruct
    public void init() {
        System.out.println("Init Method: Delivery Service Ready");
        // Initialize delivery tracking system
        // Connect to GPS service
        // Load delivery routes
    }
}
```

Example from PaymentProcessing:

```
@Component
public class TransactionLogger {
    public TransactionLogger() {
        System.out.println("TransactionLogger bean created");
    }

    @PostConstruct
    public void init() {
        System.out.println("Logger initialized");
        // Open log file
        // Connect to logging database
        // Initialize log buffer
    }
```

```
}
```

@PostConstruct Rules: - Must be `void` return type - Must have no parameters - Can be any access modifier (public, private, protected) - Must NOT be static - Can throw checked exceptions - Called AFTER all dependencies injected

🔍 @PreDestroy Deep Dive

What is @PreDestroy?

@PreDestroy marks a method to be executed BEFORE the bean is destroyed. It's the recommended way to perform cleanup logic.

Why @PreDestroy Exists:

Problem: Resource Leaks

```
@Component
public class FileProcessor {
    private FileWriter writer;

    @PostConstruct
    public void init() throws IOException {
        writer = new FileWriter("output.txt");
    }

    // ❌ PROBLEM: File never closed!
    // Resource leak when application shuts down
}
```

Solution: @PreDestroy

```
@Component
public class FileProcessor {
    private FileWriter writer;

    @PostConstruct
    public void init() throws IOException {
        writer = new FileWriter("output.txt");
    }

    @PreDestroy
    public void cleanup() throws IOException {
        if (writer != null) {
```

```
            writer.close();  // ✅ Properly closed
            System.out.println("File closed");
        }
    }
}
```

Example from FoodDelivery:

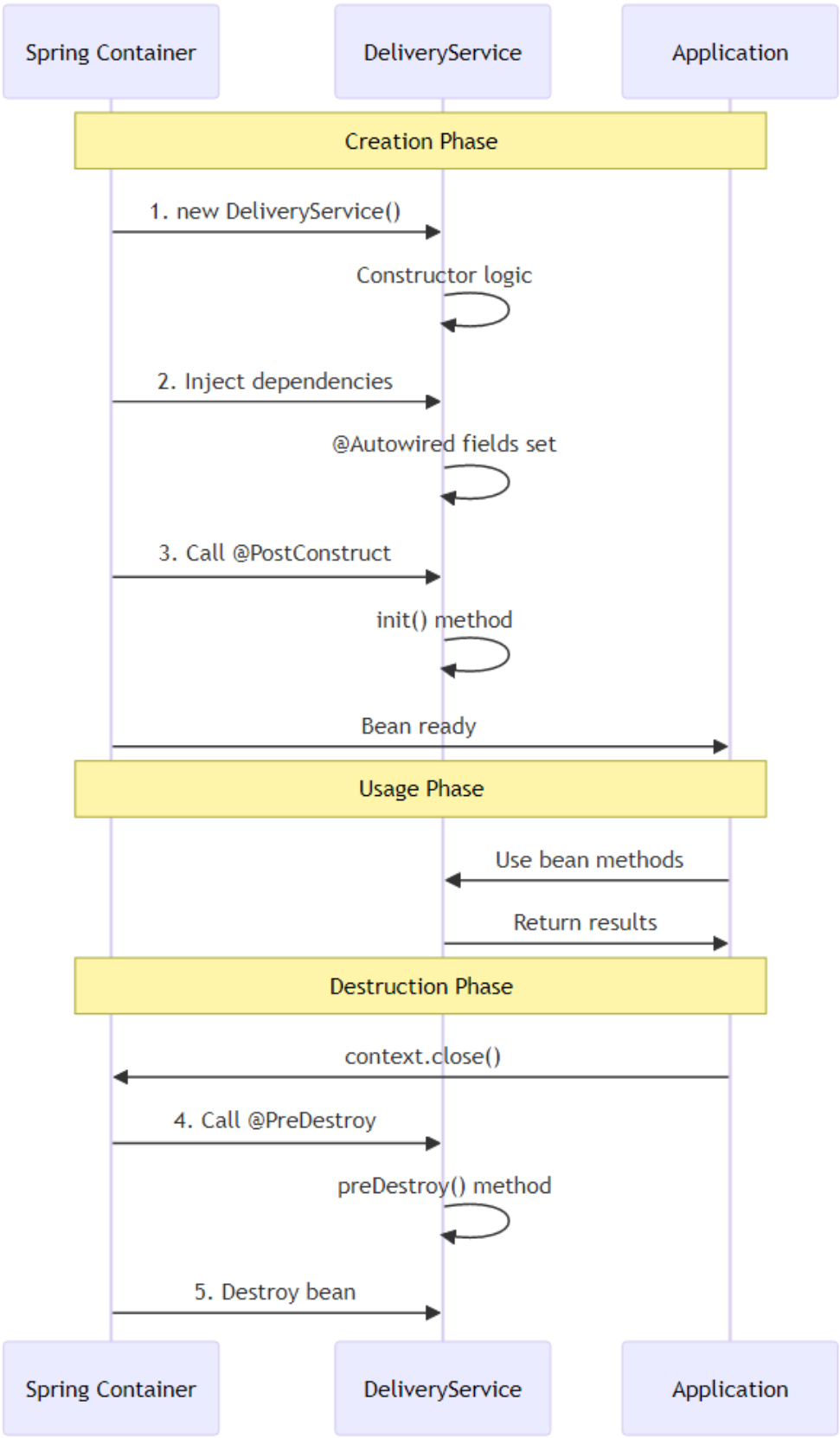
```
@Component
@Scope("singleton")
public class DeliveryService {
    @PreDestroy
    public void preDestroy() {
        System.out.println("Delivery Service Closed");
        // Close GPS connections
        // Save delivery state
        // Flush pending deliveries
    }
}
```

Example from PaymentProcessing:

```
@Component
public class TransactionLogger {
    @PreDestroy
    public void preDestroy() {
        System.out.println("Logger destroyed");
        // Flush log buffer
        // Close log file
        // Disconnect from logging database
    }
}
```

@PreDestroy Rules: - Must be `void` return type - Must have no parameters - Can be any access modifier - Must NOT be static - Can throw checked exceptions - Called BEFORE bean destruction - **NOT called for Prototype beans!**

Lifecycle Execution Order


Spring Container
DeliveryService
Application
Creation Phase
1. new DeliveryService()
Constructor logic
2. Inject dependencies
@Autowired fields set
3. Call @PostConstruct
init() method
Bean ready
Usage Phase
Use bean methods
Return results
Destruction Phase
context.close()
4. Call @PreDestroy
preDestroy() method
5. Destroy bean
Spring Container
DeliveryService
Application

Complete Example:

```java
@Component
public class CompleteLifecycleBean {
    @Autowired
    private SomeDependency dependency;

    // Phase 1: Instantiation
    public CompleteLifecycleBean() {
        System.out.println("1. Constructor called");
        System.out.println("   dependency = " + dependency);  // null
    }

    // Phase 2: Dependency Injection happens here

    // Phase 3: Initialization
    @PostConstruct
    public void init() {
        System.out.println("2. @PostConstruct called");
        System.out.println("   dependency = " + dependency);  // NOT null
        // Safe to use dependencies here
    }

    // Phase 4: Bean Ready - Application uses it

    // Phase 5: Cleanup
    @PreDestroy
    public void cleanup() {
        System.out.println("3. @PreDestroy called");
        // Cleanup resources
    }
}
```

Output:

```
1. Constructor called
   dependency = null
2. @PostConstruct called
   dependency = SomeDependency@123
3. @PreDestroy called
```

🎯 Lifecycle with Different Scopes

Singleton Scope:

```java
@Component
@Scope("singleton")
public class SingletonBean {
    @PostConstruct
    public void init() {
        System.out.println("Singleton init - called ONCE");
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("Singleton cleanup - called ONCE");
    }
}
```

Prototype Scope:

```java
@Component
@Scope("prototype")
public class PrototypeBean {
    @PostConstruct
    public void init() {
        System.out.println("Prototype init - called EVERY TIME");
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("Prototype cleanup - NEVER CALLED!");
    }
}
```

Important: @PreDestroy is NOT called for Prototype beans because Spring doesn't manage their complete lifecycle!

---

11. DEPENDENCY INJECTION PATTERNS

📌 Types of Dependency Injection

Spring supports three types of dependency injection: 1. **Constructor Injection** (Recommended ✅) 2. **Setter Injection** (For optional dependencies) 3. **Field Injection** (Not recommended ❌)

🎯 Constructor Injection (Recommended)

Definition: Dependencies are provided through the constructor.

Example from BankLoanApproval:

```
@Component
public class LoanService {
    private final LoanValidator loanValidator;

    @Autowired  // Optional for single constructor
    public LoanService(@Qualifier("incomeValidator") LoanValidator
         loanValidator) {
        this.loanValidator = loanValidator;
    }
}
```

Example from FoodDelivery:

```
@Component
public class OrderService {
    private final NotificationService notificationService;

    public OrderService(@Qualifier("smsNotification") NotificationService
         notificationService) {
        this.notificationService = notificationService;
    }
}
```

Example from PaymentProcessing:

```
@Component
public class PaymentProcessor {
    private final PaymentService paymentService;
    private final PaymentService paymentService02;

    public PaymentProcessor(
            @Qualifier("upiPayment") PaymentService paymentService,
```

```
            @Qualifier("creditCardPayment") PaymentService
          paymentService02) {
        this.paymentService = paymentService;
        this.paymentService02 = paymentService02;
    }
}
```

Advantages: - ✅ Immutability (final fields) - ✅ Required dependencies enforced - ✅ Easy to test (no reflection needed) - ✅ Null-safe (dependencies guaranteed) - ✅ Clear dependencies in constructor signature

🎯 Setter Injection

Definition: Dependencies are provided through setter methods.

Example from BankLoanApproval:

```
@Component
public class LoanService {
    private AuditService auditService;

    @Autowired
    public void setAuditService(AuditService auditService) {
        this.auditService = auditService;
    }
}
```

Example from FoodDelivery:

```
@Component
public class RestaurantService {
    private DeliveryService deliveryService;

    @Autowired
    public void setDeliveryService(DeliveryService deliveryService) {
        this.deliveryService = deliveryService;
        System.out.println("-- Setter Injection: DeliveryService injected --
          ");
    }
}
```

Advantages: - ✅ Optional dependencies - ✅ Allows reconfiguration - ✅ Circular dependency resolution - ✅ Clear setter method names

Disadvantages: - ❌ Mutable (not final) - ❌ Can be null - ❌ Dependencies not obvious

When to Use: - Optional dependencies (AuditService in BankLoanApproval) - Circular dependencies - Reconfigurable beans

🎯 Field Injection (Not Recommended)

Definition: Dependencies are injected directly into fields.

Example from PaymentProcessing:

```
@Component
public class PaymentProcessor {
    @Autowired
    private TransactionLogger transactionLogger;
}
```

Disadvantages: - ❌ Hard to test (requires reflection) - ❌ Breaks encapsulation - ❌ Hidden dependencies - ❌ Cannot make final - ❌ Tight coupling to Spring

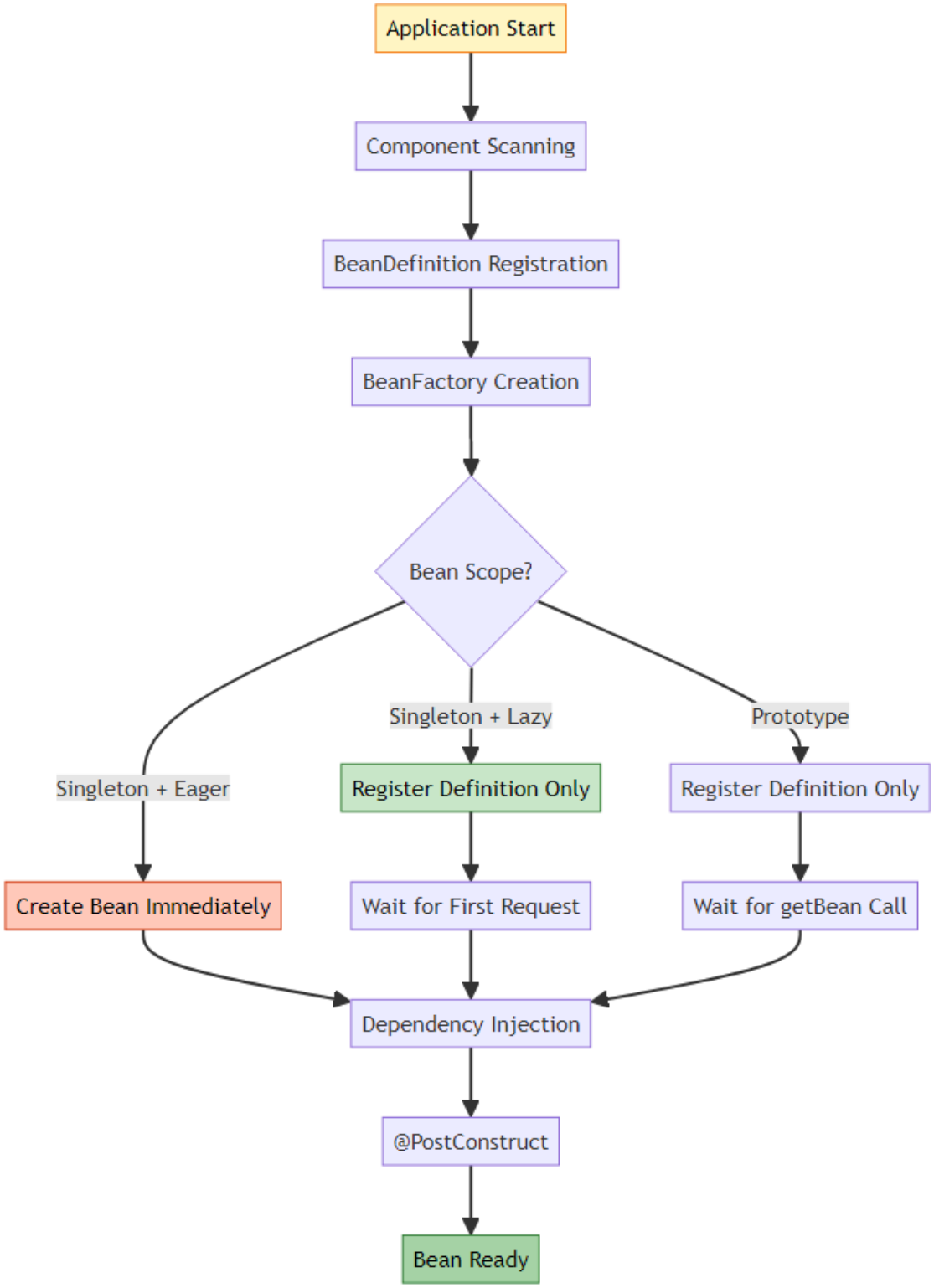
When to Use: - Quick prototyping - Legacy code - Avoid in production code

📊 Injection Types Comparison

Aspect	Constructor	Setter	Field
Immutability	✅ final fields	❌ mutable	❌ mutable
Required Dependencies	✅ enforced	❌ optional	❌ optional
Testability	✅ easy	✅ easy	❌ hard
Null Safety	✅ guaranteed	❌ can be null	❌ can be null
Circular Dependencies	❌ difficult	✅ possible	✅ possible
Visibility	✅ clear	✅ clear	❌ hidden
Recommendation	✅ Use this	⚠️ Optional only	❌ Avoid

12. INTERNAL WORKING MECHANISMS

📌 How Spring Container Works



🔍 Component Scanning Process

Step 1: @ComponentScan

```
@Configuration
@ComponentScan(basePackages = "org.example")
public class AppConfig {
}
```

What Happens: 1. Spring scans “org.example” package 2. Finds classes with @Component, @Service, @Repository, @Controller 3. Creates BeanDefinition for each 4. Registers in BeanDefinitionRegistry

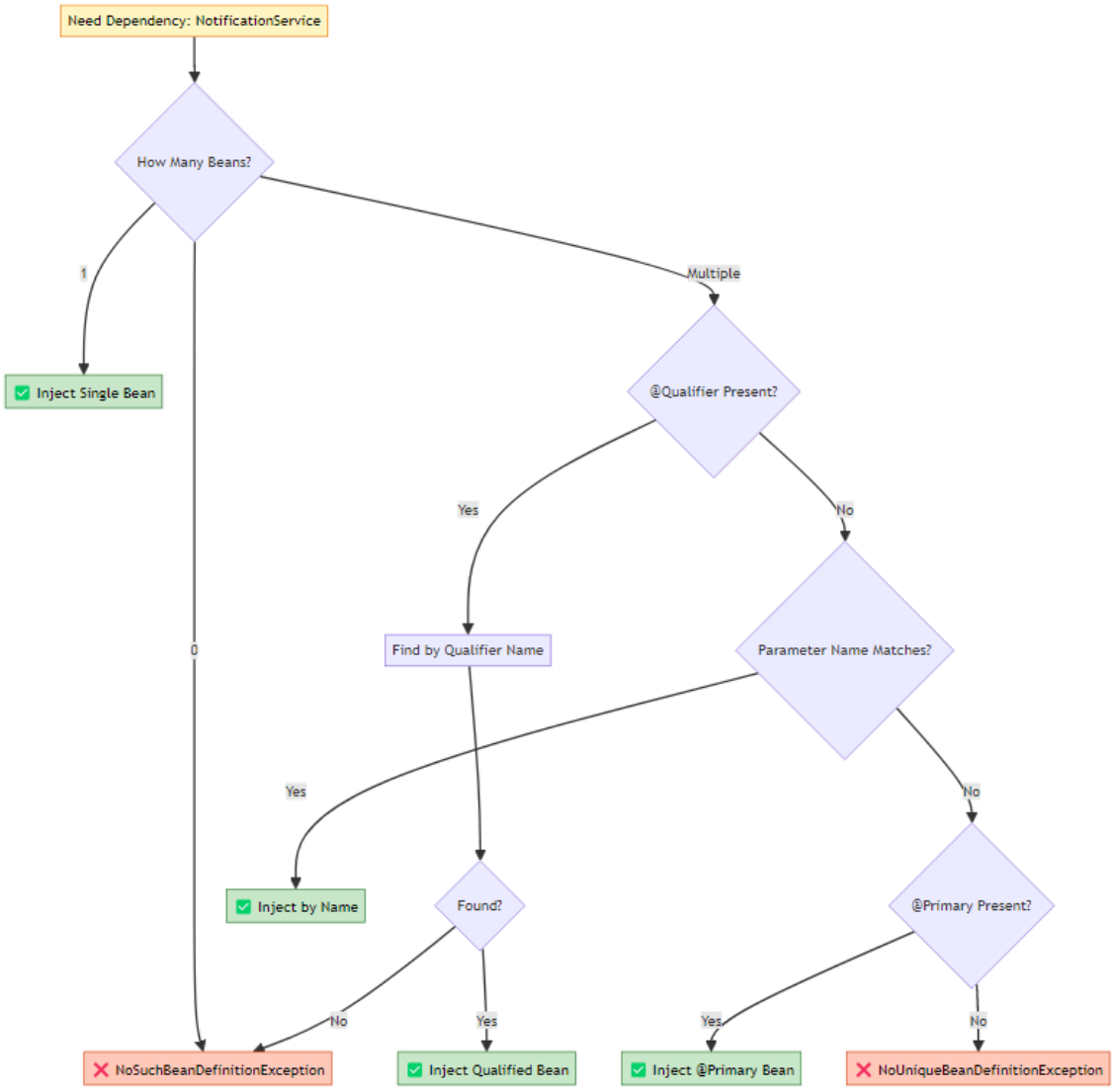
Step 2: BeanDefinition Creation

```
// Spring internally creates:
BeanDefinition def = new GenericBeanDefinition();
def.setBeanClassName("org.example.service.LoanService");
def.setScope("singleton");
def.setLazyInit(false);
def.setAutowireMode(AUTOWIRE_BY_TYPE);
```

Step 3: Bean Instantiation

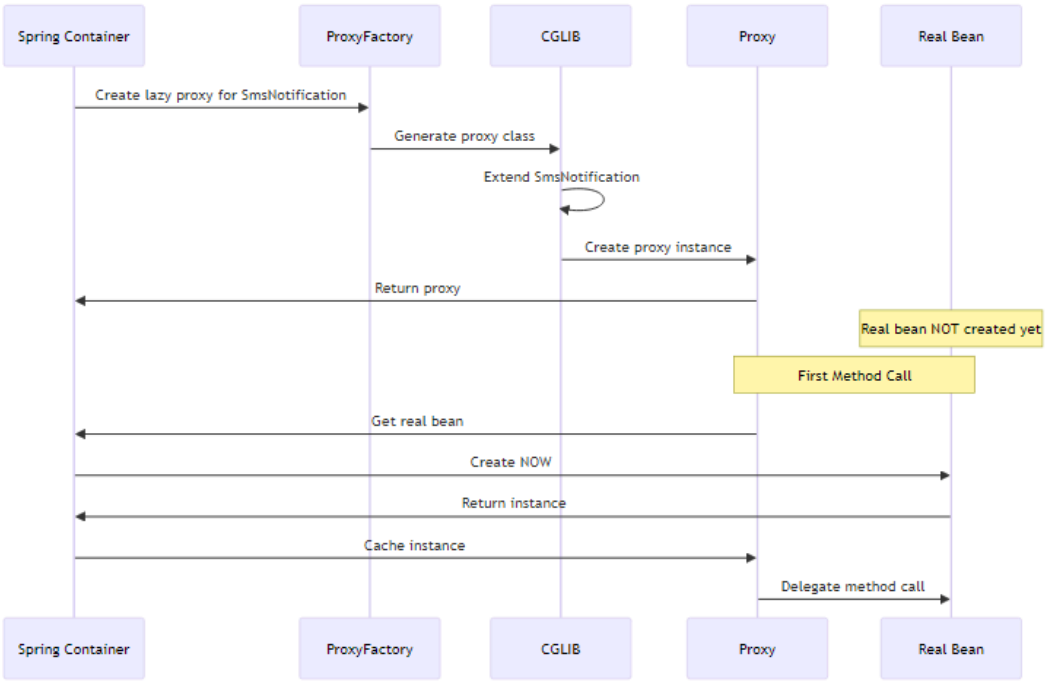
```
// Spring internally does:
Class<?> clazz = Class.forName("org.example.service.LoanService");
Constructor<?> constructor = clazz.getConstructor(LoanValidator.class);
Object bean = constructor.newInstance(loanValidator);
```

🔍 Dependency Resolution Algorithm



🔍 @Lazy Proxy Creation

How Spring Creates Lazy Proxies:



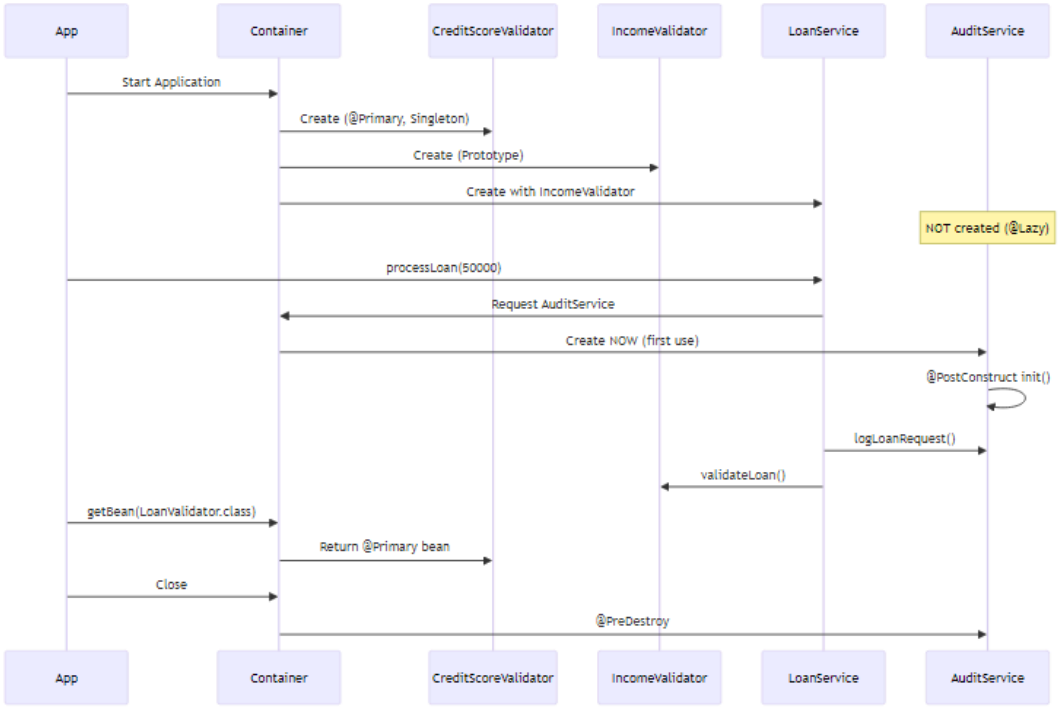
Proxy Class Structure:

```
// Spring generates something like:
public class SmsNotification$$EnhancerBySpringCGLIB$$12345678 extends
         SmsNotification {
    private SmsNotification target;
    private BeanFactory beanFactory;

    @Override
    public void sendNotification(String message) {
        if (target == null) {
            // First call - create real bean
            target = beanFactory.getBean(SmsNotification.class);
        }
        // Delegate to real bean
        target.sendNotification(message);
    }
}
```

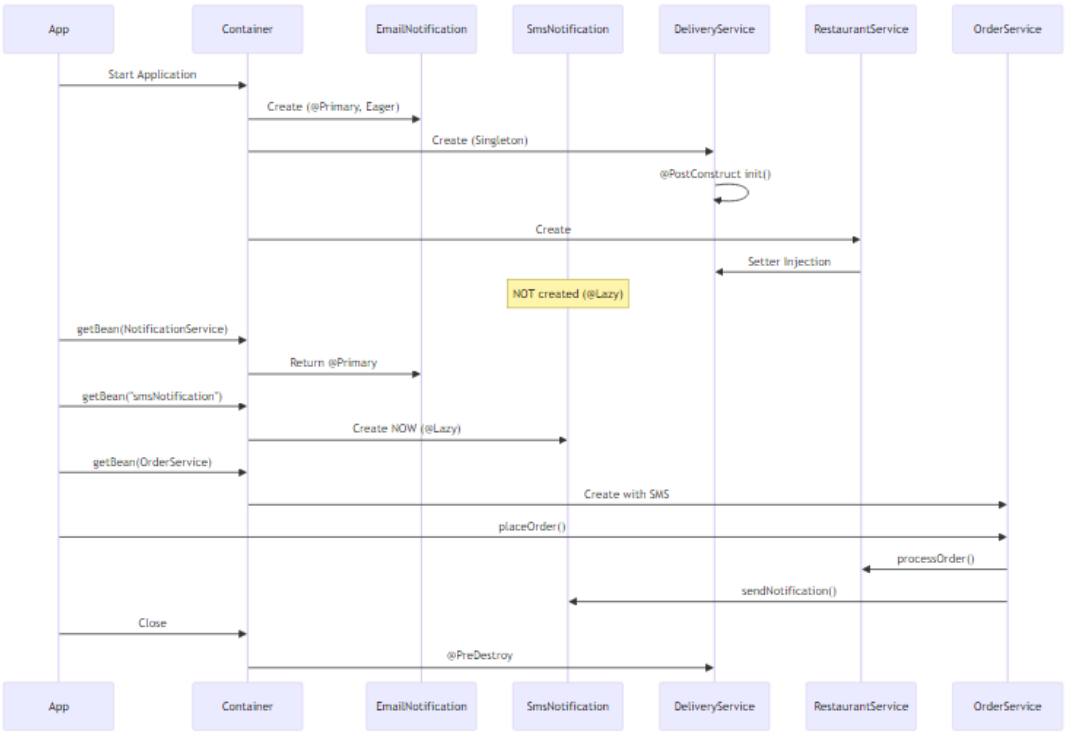
---

13. EXECUTION FLOW ANALYSIS

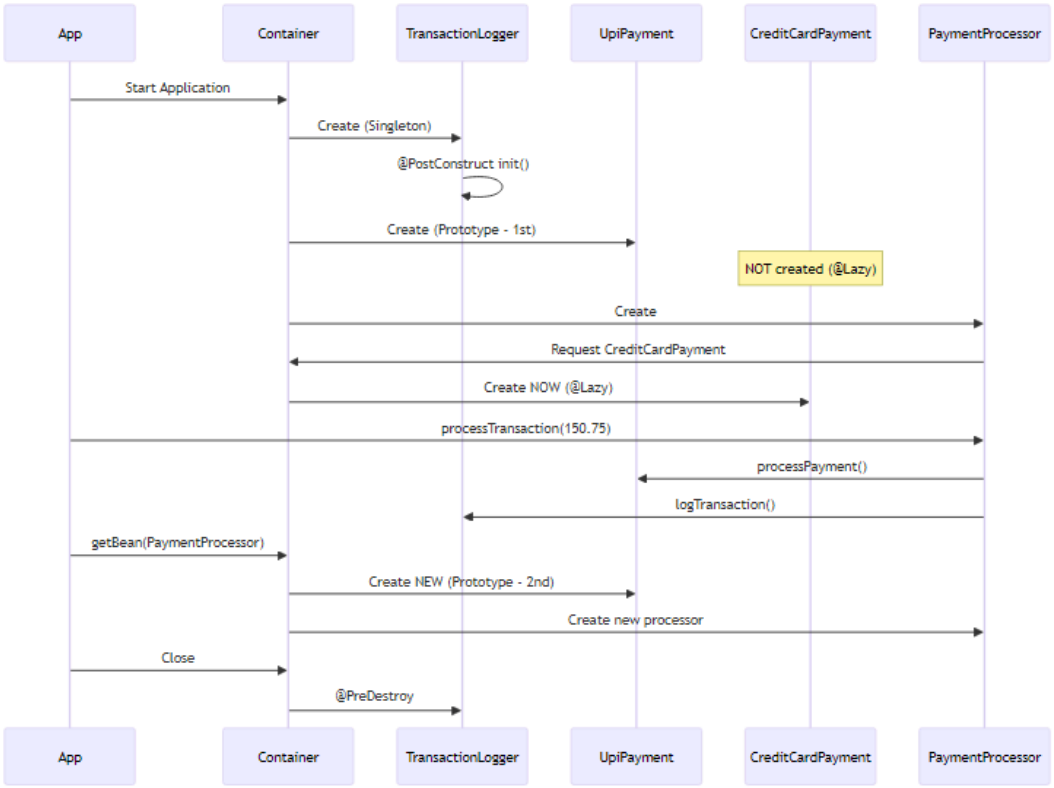
BankLoanApproval Execution Flow



FoodDelivery Execution Flow



📊 PaymentProcessing Execution Flow



---

14. REAL-WORLD APPLICATIONS

🌍 E-Commerce Platform

Scenario: Online shopping platform with multiple payment gateways

```
// Payment Gateway Interface
public interface PaymentGateway {
    boolean processPayment(Order order);
}

// Default gateway
@Primary
@Component
public class StripeGateway implements PaymentGateway {
    // Most common, default choice
}
```

```java
// Alternative gateways
@Component
public class PayPalGateway implements PaymentGateway { }

@Component
public class RazorpayGateway implements PaymentGateway { }

// Checkout Service
@Component
public class CheckoutService {
    // Uses Stripe by default (@Primary)
    public CheckoutService(PaymentGateway gateway) { }
}

// Admin Service
@Component
public class AdminPaymentService {
    // Can switch to specific gateway
    public AdminPaymentService(
        @Qualifier("payPalGateway") PaymentGateway gateway) { }
}
```

Why This Design: - @Primary for most common gateway (Stripe) - @Qualifier for specific business requirements - Easy to add new gateways - Flexible payment routing

🌍 Notification System

Scenario: Multi-channel notification system

```java
public interface NotificationChannel {
    void send(String message);
}

@Primary
@Component
public class EmailChannel implements NotificationChannel {
    // Default, most reliable
}

@Component
@Lazy
public class SmsChannel implements NotificationChannel {
```

```
    // Expensive SMS gateway
}

@Component
@Lazy
public class PushChannel implements NotificationChannel {
    // Mobile push notifications
}

@Component
public class NotificationService {
    private final NotificationChannel primary;
    private final NotificationChannel sms;

    public NotificationService(
        NotificationChannel primary,  // Email
        @Qualifier("smsChannel") NotificationChannel sms) {
        this.primary = primary;
        this.sms = sms;
    }

    public void sendUrgent(String message) {
        sms.send(message);  // SMS for urgent
    }

    public void sendNormal(String message) {
        primary.send(message);  // Email for normal
    }
}
```

Why This Design: - @Primary for default channel (Email) - @Lazy for expensive channels (SMS, Push) - Multiple channels in same service - Business logic determines channel

🌍 Database Connection Pool

Scenario: Multi-database application

```
public interface DataSource {
    Connection getConnection();
}

@Primary
@Component
```

```java
public class PrimaryDataSource implements DataSource {
    @PostConstruct
    public void init() {
        // Initialize primary database pool
    }

    @PreDestroy
    public void cleanup() {
        // Close all connections
    }
}

@Component
@Lazy
public class AnalyticsDataSource implements DataSource {
    // Expensive analytics database
    // Created only when analytics needed
}

@Component
public class UserRepository {
    // Uses primary database
    public UserRepository(DataSource dataSource) { }
}

@Component
public class AnalyticsService {
    // Uses analytics database
    public AnalyticsService(
        @Qualifier("analyticsDataSource") DataSource dataSource) { }
}
```

Why This Design: - @Primary for main database - @Lazy for analytics (expensive, rarely used) - @PostConstruct/@PreDestroy for connection management - Separate data sources for different purposes

---

15. BEST PRACTICES

✅ Dependency Injection

1. **Prefer Constructor Injection**

```java
// ✅ GOOD
@Component
public class OrderService {
    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}

// ❌ BAD
@Component
public class OrderService {
    @Autowired
    private PaymentService paymentService;
}
```

2. **Use final for Required Dependencies**

```java
// ✅ GOOD - Immutable, null-safe
private final PaymentService paymentService;

// ❌ BAD - Mutable, can be null
private PaymentService paymentService;
```

3. **Setter Injection for Optional Dependencies**

```java
// ✅ GOOD - Optional audit service
@Autowired(required = false)
public void setAuditService(AuditService auditService) {
    this.auditService = auditService;
}
```

✅ @Primary and @Qualifier

1. **Use @Primary for Default Implementation**

```java
// ✅ GOOD - Clear default
@Primary
@Component
public class EmailNotification implements NotificationService { }
```

2. **Use @Qualifier for Specific Requirements**

```java
// ✅ GOOD - Explicit selection
```

```
public OrderService(@Qualifier("smsNotification") NotificationService
        service) { }
```

3. **Only ONE @Primary per Type**

```
// ❌ BAD - Multiple @Primary
@Primary @Component
public class EmailNotification implements NotificationService { }

@Primary @Component  // ERROR!
public class SmsNotification implements NotificationService { }
```

✅ @Lazy Initialization

1. **Use @Lazy for Expensive Beans**

```
// ✅ GOOD - Expensive initialization
@Component
@Lazy
public class ReportGenerator {
    @PostConstruct
    public void init() {
        // Load heavy templates
        // Connect to reporting service
    }
}
```

2. **Combine @Primary with @Lazy**

```
// ✅ GOOD - Default but lazy
@Primary
@Component
@Lazy
public class CreditCardPayment implements PaymentService { }
```

3. **Don't Overuse @Lazy**

```
// ❌ BAD - Always-used service shouldn't be lazy
@Component
@Lazy
public class UserService { }  // Used in every request!
```

✅ Bean Scopes

1. **Use Singleton for Stateless Services**

```java
// ✅ GOOD - Stateless, thread-safe
@Component  // Singleton by default
public class EmailService {
    public void send(String message) { }
}
```

2. **Use Prototype for Stateful Objects**

```java
// ✅ GOOD - Stateful, per-request
@Component
@Scope("prototype")
public class ShoppingCart {
    private List<Item> items = new ArrayList<>();
}
```

3. **Be Careful with Singleton-Prototype Interaction**

```java
// ❌ BAD - Singleton holds Prototype
@Component
public class OrderService {
    @Autowired
    private ShoppingCart cart;  // ALWAYS SAME INSTANCE!
}

// ✅ GOOD - Get new instance each time
@Component
public class OrderService {
    @Autowired
    private ApplicationContext context;

    public void processOrder() {
        ShoppingCart cart = context.getBean(ShoppingCart.class);
    }
}
```

✅ Lifecycle Management

1. **Use @PostConstruct for Initialization**

```java
// ✅ GOOD - Dependencies available
```

```java
@PostConstruct
public void init() {
    connection.connect();
}

// ❌ BAD - Dependencies might be null
public MyService() {
    connection.connect();  // NullPointerException!
}
```

2. **Use @PreDestroy for Cleanup**

```java
// ✅ GOOD - Proper cleanup
@PreDestroy
public void cleanup() {
    if (connection != null) {
        connection.close();
    }
}
```

3. **Remember: @PreDestroy NOT Called for Prototype**

```java
@Component
@Scope("prototype")
public class PrototypeBean {
    @PreDestroy
    public void cleanup() {
        // ⚠ WARNING: This is NEVER called!
    }
}
```

---

16. COMMON PITFALLS

❌ Pitfall 1: Using Dependencies in Constructor

Problem:

```java
@Component
public class UserService {
    @Autowired
    private UserRepository repository;

    public UserService() {
        // ❌ repository is NULL here!
        repository.findAll();  // NullPointerException!
    }
}
```

Solution:

```java
@Component
public class UserService {
    @Autowired
    private UserRepository repository;

    @PostConstruct
    public void init() {
        // ✅ repository is injected
        repository.findAll();
    }
}
```

❌ Pitfall 2: Multiple @Primary

Problem:

```java
@Primary
@Component
public class EmailNotification implements NotificationService { }

@Primary  // ❌ ERROR: Only ONE @Primary allowed
@Component
public class SmsNotification implements NotificationService { }
```

Error:

```
NoUniqueBeanDefinitionException: more than one 'primary' bean found
```

Solution:

```java
@Primary
@Component
```

```
public class EmailNotification implements NotificationService { }

@Component  // ✅ No @Primary
public class SmsNotification implements NotificationService { }
```

❌ Pitfall 3: Forgetting @Qualifier Bean Name

Problem:

```
@Component
public class SmsNotification implements NotificationService { }

// ❌ Wrong bean name
public OrderService(@Qualifier("smsService") NotificationService service) {
         }
```

Error:

```
NoSuchBeanDefinitionException: No bean named 'smsService' available
```

Solution:

```
// ✅ Correct bean name (class name with lowercase first letter)
public OrderService(@Qualifier("smsNotification") NotificationService
         service) { }
```

❌ Pitfall 4: Singleton with Prototype Dependency

Problem:

```
@Component
@Scope("prototype")
public class ShoppingCart { }

@Component  // Singleton
public class OrderService {
    @Autowired
    private ShoppingCart cart;  // ❌ ALWAYS SAME INSTANCE!

    public void addItem(Item item) {
        cart.add(item);  // All users share same cart!
    }
}
```

Solution:

```
@Component
public class OrderService {
    @Autowired
    private ApplicationContext context;

    public void addItem(Item item) {
        ShoppingCart cart = context.getBean(ShoppingCart.class);
        cart.add(item);  // ✅ New cart each time
    }
}
```

❌ Pitfall 5: Expecting @PreDestroy for Prototype

Problem:

```
@Component
@Scope("prototype")
public class FileProcessor {
    private FileWriter writer;

    @PostConstruct
    public void init() throws IOException {
        writer = new FileWriter("output.txt");
    }

    @PreDestroy
    public void cleanup() throws IOException {
        writer.close();  // ❌ NEVER CALLED!
    }
}
```

Solution:

```
@Component
@Scope("prototype")
public class FileProcessor implements DisposableBean {
    private FileWriter writer;

    @PostConstruct
    public void init() throws IOException {
        writer = new FileWriter("output.txt");
    }
```

```
    // ✅ Manual cleanup
    public void close() throws IOException {
        if (writer != null) {
            writer.close();
        }
    }
}

// Usage
FileProcessor processor = context.getBean(FileProcessor.class);
try {
    processor.process();
} finally {
    processor.close();  // Manual cleanup
}
```

❌ Pitfall 6: Circular Dependencies

Problem:

```
@Component
public class A {
    @Autowired
    private B b;  // A needs B
}

@Component
public class B {
    @Autowired
    private A a;  // B needs A → Circular!
}
```

Error:

```
BeanCurrentlyInCreationException: Circular dependency
```

Solution 1: Use @Lazy

```
@Component
public class A {
    private final B b;

    public A(@Lazy B b) {  // ✅ Proxy injected
        this.b = b;
    }
```

```
}

@Component
public class B {
    private final A a;

    public B(A a) {
        this.a = a;
    }
}
```

Solution 2: Redesign (Better)

```
// Extract common functionality to separate service
@Component
public class CommonService { }

@Component
public class A {
    @Autowired
    private CommonService common;
}

@Component
public class B {
    @Autowired
    private CommonService common;
}
```

---

17. TOP INTERVIEW QUESTIONS

🎯 Dependency Injection Questions

Q1: What happens if you have multiple beans of the same type and no @Primary or @Qualifier?

Answer: Spring throws `NoUniqueBeanDefinitionException` because it cannot determine which bean to inject.

```
@Component
public class EmailNotification implements NotificationService { }

@Component
public class SmsNotification implements NotificationService { }

@Component
public class MyService {
    // ❌ ERROR: NoUniqueBeanDefinitionException
    public MyService(NotificationService service) { }
}
```

Resolution Priority: 1. @Qualifier (highest) 2. Parameter name matching 3. @Primary 4. Single bean (lowest)

Q2: Can you have multiple @Primary annotations for the same type?

Answer: No! Only ONE @Primary is allowed per bean type. If you mark multiple beans with @Primary, Spring throws an exception at startup.

```
@Primary @Component
public class EmailNotification implements NotificationService { }

@Primary @Component  // ❌ ERROR!
public class SmsNotification implements NotificationService { }

// Error: more than one 'primary' bean found
```

Q3: What's the difference between @Qualifier and parameter name matching?

Answer:

@Qualifier (Explicit):

```
public MyService(@Qualifier("smsNotification") NotificationService service)
        { }
// Explicitly selects "smsNotification" bean
```

Parameter Name Matching (Implicit):

```
public MyService(NotificationService smsNotification) { }
```

```
// Parameter name "smsNotification" matches bean name
```

Priority: @Qualifier > Parameter Name > @Primary

Q4: Why is constructor injection preferred over field injection?

Answer:

Constructor Injection Advantages: - ✅ Immutability (final fields) - ✅ Required dependencies enforced - ✅ Easy to test (no reflection) - ✅ Null-safe - ✅ Clear dependencies

Field Injection Disadvantages: - ❌ Cannot use final - ❌ Hard to test (requires reflection) - ❌ Hidden dependencies - ❌ Breaks encapsulation

```
// ✅ Constructor Injection
@Component
public class OrderService {
    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}

// ❌ Field Injection
@Component
public class OrderService {
    @Autowired
    private PaymentService paymentService;
}
```

🎯 @Lazy Questions

Q5: What happens when you inject a @Lazy bean into another bean?

Answer: Spring creates a PROXY and injects it instead of the real bean. The real bean is created only when a method is called on the proxy.

```
@Component
@Lazy
public class SmsService {
    public SmsService() {
        System.out.println("SmsService created");
```

```
    }
}

@Component
public class OrderService {
    private final SmsService smsService;

    public OrderService(SmsService smsService) {
        System.out.println("OrderService created");
        // smsService is a PROXY here
        System.out.println(smsService.getClass().getName());
        // Output: SmsService$$EnhancerBySpringCGLIB$$12345678
    }

    public void sendSms() {
        smsService.send();  // NOW real SmsService is created
    }
}
```

Output:

```
OrderService created
SmsService$$EnhancerBySpringCGLIB$$12345678
(when sendSms() is called)
SmsService created
```

Q6: Can you combine @Primary with @Lazy?

Answer: Yes! This is a powerful combination for default but expensive beans.

```
@Primary
@Component
@Lazy
public class CreditCardPayment implements PaymentService {
    // Default payment method
    // But expensive initialization
    // Created only when used
}
```

Use Case: - Credit card is the default payment method (@Primary) - But payment gateway initialization is expensive - So delay creation until actually needed (@Lazy)

Q7: What's the performance impact of @Lazy?

Answer:

Startup Time: - Without @Lazy: Slower startup (all beans created) - With @Lazy: Faster startup (beans created on demand)

First Request: - Without @Lazy: Fast (bean already created) - With @Lazy: Slower (bean created now)

Subsequent Requests: - Both: Same performance (bean cached)

Example:

```
Without @Lazy:
- Startup: 30 seconds
- First request: 10ms
- Subsequent: 10ms

With @Lazy:
- Startup: 5 seconds
- First request: 25ms (creates bean)
- Subsequent: 10ms
```

🎯 Bean Scope Questions

Q8: What's the difference between Singleton and Prototype scope?

Answer:

Aspect	Singleton	Prototype
Instances	1 per container	New per request
Creation	At startup or first use	Every getBean() call
Caching	✅ Cached	❌ Not cached
@PreDestroy	✅ Called	❌ NOT called
Thread Safety	Must be thread-safe	Each thread gets own
Use Case	Stateless services	Stateful objects

```java
// Singleton
@Component
public class EmailService {
    // One instance for entire application
}
```

```
// Prototype
@Component
@Scope("prototype")
public class ShoppingCart {
    // New instance per request
}
```

Q9: Why is @PreDestroy not called for Prototype beans?

Answer: Spring doesn't manage the complete lifecycle of Prototype beans. After creating and returning a Prototype bean, Spring "forgets" about it. The application is responsible for cleanup.

```
@Component
@Scope("prototype")
public class FileProcessor {
    @PreDestroy
    public void cleanup() {
        // ⚠ NEVER CALLED!
    }
}
```

Reason: - Prototype beans can have multiple instances - Spring doesn't track all instances - Application controls when instances are destroyed - @PreDestroy would be ambiguous (when to call?)

Solution: Manual cleanup

```
FileProcessor processor = context.getBean(FileProcessor.class);
try {
    processor.process();
} finally {
    processor.close();  // Manual cleanup
}
```

Q10: What happens when a Singleton bean has a Prototype dependency?

Answer: The Singleton bean gets ONE instance of the Prototype bean and keeps reusing it, defeating the purpose of Prototype scope!

```
@Component
@Scope("prototype")
public class ShoppingCart { }
```

```java
@Component  // Singleton
public class OrderService {
    @Autowired
    private ShoppingCart cart;  // ❌ ALWAYS SAME INSTANCE!
}
```

Problem: - OrderService is created once (Singleton) - ShoppingCart is injected once - All users share the same cart!

Solution 1: ApplicationContext

```java
@Component
public class OrderService {
    @Autowired
    private ApplicationContext context;

    public void processOrder() {
        ShoppingCart cart = context.getBean(ShoppingCart.class);
        // ✅ New cart each time
    }
}
```

Solution 2: @Lookup

```java
@Component
public abstract class OrderService {
    public void processOrder() {
        ShoppingCart cart = getCart();
        // ✅ New cart each time
    }

    @Lookup
    protected abstract ShoppingCart getCart();
}
```

🎯 Lifecycle Questions

Q11: Why can't you use dependencies in the constructor?

Answer: Dependencies are injected AFTER the constructor is called. In the constructor, all @Autowired fields are still null.

```java
@Component
public class UserService {
```

```
    @Autowired
    private UserRepository repository;

    public UserService() {
        System.out.println("Constructor: " + repository);
        // Output: Constructor: null

        // ❌ NullPointerException!
        repository.findAll();
    }

    @PostConstruct
    public void init() {
        System.out.println("PostConstruct: " + repository);
        // Output: PostConstruct: UserRepository@123

        // ✅ Safe to use
        repository.findAll();
    }
}
```

Lifecycle Order: 1. Constructor called (dependencies are null) 2. Dependencies injected 3. @PostConstruct called (dependencies available)

Q12: What's the execution order of lifecycle methods?

Answer:

```
@Component
public class CompleteLifecycleBean implements InitializingBean,
         DisposableBean {

    // 1. Constructor
    public CompleteLifecycleBean() {
        System.out.println("1. Constructor");
    }

    // 2. @PostConstruct
    @PostConstruct
    public void postConstruct() {
        System.out.println("2. @PostConstruct");
    }

    // 3. InitializingBean.afterPropertiesSet()
    @Override
```

```
    public void afterPropertiesSet() {
        System.out.println("3. afterPropertiesSet");
    }

    // 4. Custom init-method (if configured)
    public void customInit() {
        System.out.println("4. customInit");
    }

    // 5. @PreDestroy
    @PreDestroy
    public void preDestroy() {
        System.out.println("5. @PreDestroy");
    }

    // 6. DisposableBean.destroy()
    @Override
    public void destroy() {
        System.out.println("6. destroy");
    }

    // 7. Custom destroy-method (if configured)
    public void customDestroy() {
        System.out.println("7. customDestroy");
    }
}
```

Output:

```
1. Constructor
2. @PostConstruct
3. afterPropertiesSet
4. customInit
5. @PreDestroy
6. destroy
7. customDestroy
```

Recommendation: Use @PostConstruct and @PreDestroy (simplest and standard)

🎯 Advanced Questions

Q13: How does Spring resolve circular dependencies?

Answer: Spring uses a three-level cache mechanism and proxy creation to resolve circular dependencies.

Problem:

```
@Component
public class A {
    @Autowired
    private B b;  // A needs B
}

@Component
public class B {
    @Autowired
    private A a;  // B needs A
}
```

Spring's Solution: 1. Create A (partially initialized) 2. Put A in "early reference" cache 3. Start injecting dependencies into A 4. Need B, so create B 5. B needs A, get A from early reference cache 6. Inject A into B (B is complete) 7. Inject B into A (A is complete)

With Constructor Injection (Fails):

```
@Component
public class A {
    public A(B b) { }  // ✗ Cannot resolve
}

@Component
public class B {
    public B(A a) { }  // ✗ Cannot resolve
}
```

Why it fails: Spring cannot create A without B, and cannot create B without A.

Solution: Use @Lazy

```
@Component
public class A {
    public A(@Lazy B b) { }  // ✓ Proxy injected
}

@Component
public class B {
```

```
    public B(A a) { }
}
```

Q14: What's the difference between @Component and @Bean?

Answer:

@Component: - Class-level annotation - Auto-detected by component scanning - Spring creates bean automatically - Used for your own classes

```
@Component
public class EmailService {
    // Spring creates this automatically
}
```

@Bean: - Method-level annotation - Used in @Configuration classes - Manual bean creation - Used for third-party classes

```
@Configuration
public class AppConfig {
    @Bean
    public DataSource dataSource() {
        // Manual creation for third-party class
        return new HikariDataSource();
    }
}
```

Aspect	@Component	@Bean
Level	Class	Method
Detection	Auto (component scan)	Manual (@Configuration)
Use Case	Your classes	Third-party classes
Control	Less control	Full control
Customization	Limited	Flexible

Q15: Can you inject a List or Map of beans?

Answer: Yes! Spring can inject all beans of a type into a List or Map.

List Injection:

```
public interface NotificationService {
    void send(String message);
```

```
}

@Component
public class EmailNotification implements NotificationService { }

@Component
public class SmsNotification implements NotificationService { }

@Component
public class PushNotification implements NotificationService { }

@Component
public class NotificationManager {
    private final List<NotificationService> allNotifications;

    // Injects ALL NotificationService beans
    public NotificationManager(List<NotificationService> allNotifications) {
        this.allNotifications = allNotifications;
        // List contains: [Email, SMS, Push]
    }

    public void sendToAll(String message) {
        allNotifications.forEach(n -> n.send(message));
    }
}
```

Map Injection:

```
@Component
public class NotificationManager {
    private final Map<String, NotificationService> notificationMap;

    // Key = bean name, Value = bean instance
    public NotificationManager(Map<String, NotificationService>
         notificationMap) {
        this.notificationMap = notificationMap;
        // Map: {
        //   "emailNotification" -> EmailNotification,
        //   "smsNotification" -> SmsNotification,
        //   "pushNotification" -> PushNotification
        // }
    }

    public void send(String channel, String message) {
        NotificationService service = notificationMap.get(channel +
         "Notification");
```

```
        service.send(message);
    }
}
```

Use Cases: - Strategy pattern (select from multiple implementations) - Plugin architecture - Dynamic service selection - Broadcast to all implementations

Q16: What happens if @PostConstruct throws an exception?

Answer: The bean creation fails, and Spring throws a `BeanCreationException`. The bean is NOT added to the container.

```
@Component
public class DatabaseService {
    @PostConstruct
    public void init() {
        throw new RuntimeException("Database connection failed");
    }
}
```

Result: - Application startup fails - Bean is not created - Other beans depending on this bean also fail - Fail-fast behavior (good for catching errors early)

Best Practice:

```
@Component
public class DatabaseService {
    @PostConstruct
    public void init() {
        try {
            connectToDatabase();
        } catch (Exception e) {
            // Log error
            logger.error("Failed to connect to database", e);
            // Rethrow to fail fast
            throw new BeanCreationException("Database initialization
         failed", e);
        }
    }
}
```

🎓 CONCLUSION

📚 What You've Learned

Through these three real-world case studies, you've mastered:

✅ **Dependency Injection Patterns** - Constructor injection for required dependencies - Setter injection for optional dependencies - Field injection (and why to avoid it)

✅ **Dependency Resolution** - @Primary for default implementations - @Qualifier for explicit selection - Resolution priority and ambiguity handling

✅ **Performance Optimization** - @Lazy for expensive beans - Lazy proxy mechanism - Startup time vs first-request tradeoff

✅ **Bean Scopes** - Singleton for stateless services - Prototype for stateful objects - Scope interaction gotchas

✅ **Lifecycle Management** - @PostConstruct for initialization - @PreDestroy for cleanup - Resource management best practices

✅ **Real-World Applications** - Bank loan approval system - Food delivery platform - Payment processing gateway

🚀 Next Steps

1. **Practice:** Build your own case studies
2. **Experiment:** Try different combinations of annotations
3. **Debug:** Use Spring's logging to understand bean creation
4. **Optimize:** Profile your application and apply @Lazy strategically
5. **Test:** Write unit tests for your Spring beans

📧 Contact

Questions or feedback? Reach out!

Avinash Dhanuka

GITHUB AVINASH-706 | EMAIL AVUNASHDHANUKA@GMAIL.COM

Happy Coding!

Master Spring, Build Better Applications